

ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



## ĐÁNH GIÁ VÀ KIỂM ĐỊNH CHẤT LƯỢNG PHẦN MỀM

Đề tài: “Thiết kế và Phát triển Hệ thống Tự động hóa Kiểm thử Đa cấp độ”

Giảng viên hướng dẫn:

TS. Trịnh Thanh Bình

Sinh viên thực hiện:

Nguyễn Văn Nhật (23010887)

Nguyễn Thị Kiều Trinh (23010632)

Lê Thị Kiều Trang (23010502)

Quách Hữu Nam (23012358)

Lớp tín chỉ: Đánh giá và kiểm định chất lượng phần mềm  
(CSE703010-1-3-25) (COUR01.LT5)

Chương trình đào tạo: Công nghệ thông tin

Hà Nội, tháng 06 năm 2026

## **LỜI CẢM ƠN**

Chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới TS. Trịnh Thanh Bình, giảng viên hướng dẫn đã tận tình chỉ bảo, truyền đạt những kiến thức quý báu và đưa ra những góp ý chuyên môn trong suốt quá trình thực hiện đề tài “Thiết kế và Phát triển Hệ thống Tự động hóa Kiểm thử Đa cấp độ”.

## MỤC LỤC

|                                                             |    |
|-------------------------------------------------------------|----|
| LỜI CẢM ƠN .....                                            | 1  |
| DANH MỤC BẢNG BIỂU.....                                     | 5  |
| BẢNG PHÂN CHIA CÔNG VIỆC .....                              | 7  |
| 1.1. LÝ DO CHỌN ĐỀ TÀI.....                                 | 8  |
| 1.2. MỤC TIÊU CỦA ĐỀ TÀI.....                               | 9  |
| 1.2.1 Mục tiêu tổng quát.....                               | 9  |
| 1.2.2 Mục tiêu cụ thể .....                                 | 9  |
| 1.2.3 Phạm vi không bao gồm .....                           | 10 |
| 1.3 TỪ ĐIỂN THUẬT NGỮ .....                                 | 11 |
| 1.4 TỔNG QUÁT .....                                         | 13 |
| 1.4.1 Tổng quát hệ thống.....                               | 13 |
| 1.4.2 Cấu trúc tài liệu .....                               | 14 |
| 1.5 CÁC TÁC NHÂN .....                                      | 14 |
| 1.6 CÁC CHỨC NĂNG CỦA HỆ THỐNG .....                        | 16 |
| 1.7 SƠ ĐỒ CHỨC NĂNG .....                                   | 18 |
| 1.8 BIỂU ĐỒ USE – CASE TỔNG QUAN.....                       | 18 |
| 1.9 BIỂU ĐỒ USE – CASE PHÂN RÃ.....                         | 19 |
| 1.9.1 Sự tương tác của các tác nhân với hệ thống .....      | 19 |
| 1.9.2 Các use – case đối với người dùng.....                | 19 |
| 1.10 QUY TRÌNH NGHIỆP VỤ.....                               | 20 |
| 1.11 ĐẶC TẢ USE – CASE .....                                | 21 |
| 1.11.1 Khởi tạo / Mở dự án (Workspace).....                 | 21 |
| 1.11.2 Tạo và soạn thảo Test Case .....                     | 24 |
| 1.11.3 Thực thi kiểm thử (Execute Test Case) .....          | 27 |
| 1.11.4 Gỡ lỗi kiểm thử theo từng bước (Debug Mode) .....    | 30 |
| 1.11.5 Quản lý và thực thi Test Suite .....                 | 32 |
| 1.11.6 Soạn thảo và gửi API Request.....                    | 35 |
| 1.11.7 Quản lý Object Repository (Element Locators) .....   | 38 |
| 1.11.8 Quản lý môi trường và Profile biến .....             | 41 |
| 1.11.9 Quản lý Custom Keyword.....                          | 44 |
| 1.11.10 Quản lý Data File (Kiểm thử hướng dữ liệu) .....    | 46 |
| 1.11.11 AI Agent – Hỗ trợ sinh và chỉnh sửa Test Case ..... | 48 |
| 1.11.12 Kiểm thử ứng dụng Mobile (Appium Inspector) .....   | 51 |
| 1.11.13 Xem báo cáo và lịch sử chạy kiểm thử.....           | 54 |
| 1.12 CÁC YÊU CẦU PHI CHỨC NĂNG .....                        | 57 |
| 1.13 TÍNH BẢO MẬT.....                                      | 57 |

|                                                                |            |
|----------------------------------------------------------------|------------|
| 3.14 TÍNH RÀNG BUỘC .....                                      | 58         |
| 3.15 HIỆU NĂNG (PERFORMANCE) .....                             | 58         |
| 3.16 KHẢ NĂNG TƯƠNG THÍCH (COMPATIBILITY) .....                | 60         |
| <b>PHẦN II: PHÂN TÍCH HỆ THỐNG .....</b>                       | <b>63</b>  |
| 2.1 TỔNG QUAN SẢN PHẨM .....                                   | 63         |
| 2.2 KIẾN TRÚC TỔNG THỂ .....                                   | 64         |
| 2.2.2 Kiến trúc Electron .....                                 | 64         |
| 2.2.3 Kiến trúc Feature theo Vertical Slice .....              | 65         |
| 2.3 CÔNG NGHỆ SỬ DỤNG .....                                    | 74         |
| 2.4 PHẠM VI KIỂM THỬ .....                                     | 77         |
| 2.4.1 Những chức năng được kiểm thử .....                      | 77         |
| 2.4.2 Những chức năng chưa được kiểm thử .....                 | 78         |
| 2.5 CHIẾN LƯỢC KIỂM THỬ .....                                  | 79         |
| 2.5.1 Kiểm thử đa nền tảng .....                               | 79         |
| 2.5.2 Kiểm thử đơn vị .....                                    | 80         |
| 2.5.3 Kiểm thử tích hợp .....                                  | 81         |
| 2.5.4 Kiểm thử hệ thống .....                                  | 82         |
| 2.5.5 Kiểm thử hồi quy .....                                   | 83         |
| 2.5.6 Kiểm thử hiệu năng .....                                 | 85         |
| 2.5.7 Kiểm thử bảo mật .....                                   | 86         |
| 2.6 ĐIỀU KIỆN CHẤP NHẬN .....                                  | 87         |
| 2.6.1 Tiêu chí hoàn thành kiểm thử .....                       | 87         |
| 2.6.2 Tiêu chí chấp nhận sản phẩm .....                        | 88         |
| 2.7 PHÂN LOẠI LỖI .....                                        | 88         |
| <b>PHẦN III: THỰC HIỆN KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG .....</b> | <b>91</b>  |
| 3.1 CÀI ĐẶT MÔI TRƯỜNG KIỂM THỬ .....                          | 91         |
| 3.1.1 Tổng quan môi trường kiểm thử .....                      | 91         |
| 3.1.2 Yêu cầu môi trường phần mềm .....                        | 92         |
| 3.1.3 Yêu cầu phần cứng .....                                  | 93         |
| 3.2 KIỂM THỬ HỆ THỐNG KIỂM THỬ .....                           | 93         |
| 3.2.1 Khởi tạo dự án kiểm thử .....                            | 94         |
| 3.2.2 Cấu hình môi trường thực thi .....                       | 95         |
| 3.2.3 Xây dựng Test Case .....                                 | 96         |
| 3.2.4 Thiết kế Test Suite .....                                | 97         |
| 3.2.5 AI Agent hỗ trợ sinh kiểm thử .....                      | 99         |
| 3.2.6 Báo cáo kiểm thử .....                                   | 100        |
| <b>PHẦN IV: BÁO CÁO PHÂN TÍCH TỔNG HỢP .....</b>               | <b>101</b> |

|                                               |            |
|-----------------------------------------------|------------|
| <b>4.1 TỔNG HỢP KẾT QUẢ KIỂM THỬ .....</b>    | <b>101</b> |
| <b>4.2 ĐÁNH GIÁ CHẤT LƯỢNG HỆ THỐNG .....</b> | <b>101</b> |
| <b>4.3 CẤU TRÚC MÃ NGUỒN .....</b>            | <b>102</b> |
| <b>4.4 KẾT LUẬN .....</b>                     | <b>102</b> |
| <b>TÀI LIỆU THAM KHẢO.....</b>                | <b>103</b> |

## DANH MỤC BẢNG BIỂU

|                                                                                        |     |
|----------------------------------------------------------------------------------------|-----|
| Bảng 1.1 – Bảng tổng hợp các tác nhân của hệ thống JKAUTO.....                         | 16  |
| Bảng 1.2 – Bảng đặc tả use-case Khởi tạo / Mở dự án (Workspace) .....                  | 23  |
| Bảng 1.3 – Dữ liệu chức năng Khởi tạo / Mở dự án (Workspace).....                      | 23  |
| Bảng 1.4 – Bảng đặc tả use-case Tạo và soạn thảo Test Case.....                        | 25  |
| Bảng 1.5 – Dữ liệu chức năng Tạo và soạn thảo Test Case .....                          | 27  |
| Bảng 1.6 – Bảng đặc tả use-case Thực thi kiểm thử (Execute Test Case) .....            | 29  |
| Bảng 1.7 – Dữ liệu chức năng Thực thi kiểm thử (Execute Test Case) .....               | 30  |
| Bảng 1.8 – Bảng đặc tả use-case Gỡ lỗi kiểm thử theo từng bước (Debug Mode)<br>.....   | 31  |
| Bảng 1.9 – Dữ liệu chức năng Gỡ lỗi kiểm thử theo từng bước (Debug Mode).....          | 32  |
| Bảng 1.10 – Bảng đặc tả use-case Quản lý và thực thi Test Suite.....                   | 34  |
| Bảng 1.11 – Dữ liệu chức năng Quản lý và thực thi Test Suite .....                     | 35  |
| Bảng 1.12 – Bảng đặc tả use-case Soạn thảo và gửi API Request .....                    | 37  |
| Bảng 1.13 – Dữ liệu chức năng Soạn thảo và gửi API Request .....                       | 38  |
| Bảng 1.14 – Bảng đặc tả use-case Quản lý Object Repository (Element<br>Locators) ..... | 40  |
| Bảng 1.15 – Dữ liệu chức năng Quản lý Object Repository (Element Locators)<br>.....    | 41  |
| Bảng 1.16 – Bảng đặc tả use-case Quản lý môi trường và Profile biến .....              | 42  |
| Bảng 1.17 – Dữ liệu chức năng Quản lý môi trường và Profile biến .....                 | 44  |
| Bảng 1.18 – Bảng đặc tả use-case Quản lý Custom Keyword .....                          | 45  |
| Bảng 1.19 – Dữ liệu chức năng Quản lý Custom Keyword .....                             | 46  |
| Bảng 1.20 – Dữ liệu chức năng Quản lý Data File (Kiểm thử hướng dữ liệu)..             | 48  |
| Bảng 1.21 – Bảng đặc tả use-case AI Agent – Hỗ trợ sinh và chỉnh sửa Test<br>Case..... | 50  |
| Bảng 1.22 – Dữ liệu chức năng AI Agent – Hỗ trợ sinh và chỉnh sửa Test Case<br>.....   | 51  |
| Bảng 1.23 – Bảng đặc tả use-case Kiểm thử ứng dụng Mobile (Appium<br>Inspector) .....  | 53  |
| Bảng 1.24 – Dữ liệu chức năng Kiểm thử ứng dụng Mobile (Appium Inspector)<br>.....     | 54  |
| Bảng 1.25 – Bảng đặc tả use-case Xem báo cáo và lịch sử chạy kiểm thử .....            | 56  |
| Bảng 1.26 – Dữ liệu chức năng Xem báo cáo và lịch sử chạy kiểm thử .....               | 57  |
| Bảng 1.27 – Chỉ tiêu hiệu năng của hệ thống JKAUTO.....                                | 60  |
| Bảng 1.28 – Ma trận tương thích hệ điều hành của JKAUTO.....                           | 62  |
| Bảng 3.1. Các thành phần trong môi trường kiểm thử .....                               | 91  |
| Bảng 4.2. Đánh giá chất lượng hệ thống .....                                           | 101 |

## DANH MỤC HÌNH ẢNH

|                                                                                              |     |
|----------------------------------------------------------------------------------------------|-----|
| Hình 1.1 – Biểu đồ Use-Case tổng quan tương tác giữa các tác nhân và hệ thống Phần mềm ..... | 19  |
| Hình 1.2 – Biểu đồ thể hiện sự tương tác của các tác nhân với hệ thống Phần mềm .....        | 19  |
| Hình 1.3 – Biểu đồ Use-Case phân rã các luồng nghiệp vụ đối với người dùng .....             | 20  |
| Hình 1.4 – Sơ đồ hoạt động quy trình nghiệp vụ kiểm thử tự động trên nền tảng Phần mềm ..... | 21  |
| Hình 2.1 — Sơ đồ phụ thuộc giữa các package trong monorepo JKAuto.....                       | 64  |
| Hình 2.2 — Kiến trúc ba tầng Electron và ranh giới IPC.....                                  | 65  |
| Hình 2.3 : Minh họa chức năng Project .....                                                  | 67  |
| Hình 2.4 : Minh họa chức năng Explorer .....                                                 | 68  |
| Hình 2.5 : Minh họa chức năng Test Cases .....                                               | 69  |
| Hình 2.6 : Minh họa chức năng API Request .....                                              | 70  |
| Hình 2.7 : Minh họa chức năng Test Suites.....                                               | 71  |
| Hình 2.8 : Minh họa chức năng Keywords.....                                                  | 72  |
| Hình 3.1. Hộp thoại tạo dự án kiểm thử mới trong JKAuto. ....                                | 95  |
| Hình 3.2. Cấu trúc dự án sau khi được khởi tạo trong Explorer.....                           | 95  |
| Hình 3.3. Giao diện cấu hình Environment Profile. ....                                       | 95  |
| Hình 3.4. Giao diện Test Case Editor. ....                                                   | 97  |
| Hình 3.5. Giao diện Test Suite Editor.....                                                   | 98  |
| Hình 3.6. Giao diện AI Agent Chat hỗ trợ phân tích dự án và sinh kiểm thử. ..                | 99  |
| Hình 3.7. Giao diện Reports hiển thị kết quả kiểm thử. ....                                  | 100 |
| Hình 4.1. Dashboard tổng hợp kết quả kiểm thử. ....                                          | 101 |

## BẢNG PHÂN CHIA CÔNG VIỆC

| STT | Họ và tên                                 | MSV      | Nhiệm vụ                                                                                                                                                                                                                               | Đánh giá |
|-----|-------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 1   | Nguyễn Văn Nhật<br>( <i>Trưởng nhóm</i> ) | 23010887 | Phân tích và thiết kế kiến trúc hệ thống JKAuto (Phần II); xây dựng môi trường thực nghiệm, triển khai kiểm thử tự động và tích hợp CI/CD (Phần III); tổng hợp kết quả, hoàn thiện báo cáo và điều phối các nội dung của đề tài.       | 10       |
| 2   | Nguyễn Thị Kiều Trinh                     | 23010632 | Nghiên cứu cơ sở lý thuyết về kiểm thử phần mềm và tiêu chuẩn ISO/IEC 25010 (Phần I); thiết kế các Test Case kiểm thử giao diện người dùng (UI E2E); đánh giá các tiêu chí về tính phù hợp chức năng và khả năng sử dụng của hệ thống. | 10       |
| 3   | Quách Hữu Nam                             | 23012358 | Thiết kế và thực hiện kiểm thử API (Phần III); nghiên cứu Engine, Keyword Registry và khả năng tương thích đa nền tảng của JKAuto (Phần II); xây dựng ma trận Test Case và hỗ trợ phân tích kết quả kiểm thử.                          | 10       |
| 4   | Lê Thị Kiều Trang                         | 23010502 | Đánh giá kết quả kiểm thử theo tiêu chuẩn ISO/IEC 25010 (Phần IV); rà soát nội dung, chuẩn hóa định dạng báo cáo; tổng hợp nhận xét và đề xuất các hướng cải tiến cho hệ thống.                                                        | 10       |



## PHẦN I: TỔNG QUAN ĐỀ TÀI VÀ CƠ SỞ ĐÁNH GIÁ

### 1.1. Lý do chọn đề tài

Trong kỷ nguyên chuyển đổi số, phần mềm đã phát triển thành những hệ sinh thái phức tạp, đa nền tảng từ ứng dụng mạng, máy tính để bàn đến thiết bị di động và các dịch vụ giao diện lập trình ứng dụng. Sự bùng nổ này đặt ra thách thức lớn cho công tác kiểm thử và đảm bảo chất lượng phần mềm, chót chặn quan trọng quyết định sự thành bại của sản phẩm công nghệ.

Thực tế hiện nay, thị trường công cụ kiểm thử tự động đang tồn tại một khoảng cách lớn. Các khung kiểm thử hướng mã nguồn tuy mạnh mẽ nhưng đòi hỏi kỹ sư phải có tư duy lập trình vững chắc, tạo rào cản lớn cho kiểm thử viên thủ công. Ngược lại, các công cụ trực quan không mã nguồn tuy dễ tiếp cận nhưng lại hạn chế về khả năng mở rộng, khó quản lý phiên bản và thiếu linh hoạt khi xử lý logic phức tạp. Bên cạnh đó, việc tích hợp Trí tuệ nhân tạo để tự động hóa kịch bản và sửa lỗi tuy là xu hướng tất yếu nhưng chưa có nhiều giải pháp tối ưu hóa liền mạch.

Từ thực trạng đó, việc nghiên cứu và kiểm định một hệ thống hỗ trợ kiểm thử tự động đa nền tảng, tích hợp trí tuệ nhân tạo và lưu trữ cấu trúc dữ liệu mở như JSON hoặc YAML là một đề tài mang tính cấp thiết. Đề tài này được lựa chọn dựa trên những cơ sở cốt lõi về tính toàn diện và ý nghĩa thực tiễn cao.

Trước hết, một công cụ kiểm thử luôn đòi hỏi độ tin cậy tuyệt đối. Việc chọn đề tài này cho phép áp dụng toàn diện các kiến thức của học phần Đánh giá và kiểm định chất lượng phần mềm, từ kiểm định tính đúng đắn của schema dữ liệu, đánh giá hiệu năng kiến trúc đa tiến trình, đến kiểm thử các luồng nghiệp vụ phức tạp chạy song song.

Tiếp theo, việc nghiên cứu một hệ thống có định hướng lưu trữ tài nguyên thân thiện với các công cụ quản lý mã nguồn giúp người thực hiện hiểu rõ cách đánh giá khả năng bảo trì và khả năng mở rộng theo tiêu chuẩn quốc tế ISO 25010.

Cuối cùng, kết quả nghiên cứu góp phần đưa ra giải pháp thu hẹp khoảng cách giữa kiểm thử trực quan và kiểm thử lập trình, từ đó nâng cao hiệu suất hoạt động cho các đội ngũ phát triển phần mềm hiện đại.

## **1.2. Mục tiêu của đề tài**

### **1.2.1 Mục tiêu tổng quát**

Đề tài được thực hiện nhằm nghiên cứu, phân tích và đánh giá chất lượng của một hệ thống hỗ trợ kiểm thử tự động đa nền tảng đang trong quá trình phát triển. Thông qua việc khảo sát kiến trúc hệ thống, mô hình dữ liệu, quy trình vận hành và các chức năng cốt lõi, đề tài hướng đến việc xây dựng một quy trình kiểm định chất lượng phù hợp, giúp đánh giá mức độ đáp ứng các yêu cầu kỹ thuật cũng như khả năng vận hành thực tế của sản phẩm.

Bên cạnh đó, đề tài tập trung xác định những ưu điểm, hạn chế, rủi ro tiềm ẩn và các vấn đề có thể ảnh hưởng đến độ tin cậy của hệ thống trong quá trình triển khai. Trên cơ sở đó, đề xuất các giải pháp cải tiến nhằm nâng cao chất lượng phần mềm, tăng khả năng mở rộng, bảo trì, kiểm thử và phát hành sản phẩm trong tương lai. Kết quả nghiên cứu góp phần cung cấp cơ sở khoa học và thực tiễn cho việc đánh giá mức độ sẵn sàng của hệ thống trước khi đưa vào sử dụng trong môi trường thực tế.

### **1.2.2 Mục tiêu cụ thể**

Để đạt được mục tiêu tổng quát, đề tài tập trung nghiên cứu, phân tích và đánh giá các thành phần chức năng chính của hệ thống phần mềm kiểm thử tự động đang được phát triển. Các nội dung nghiên cứu bao gồm:

- Phân tích cơ chế quản lý không gian làm việc, cấu trúc dự án kiểm thử, tổ chức dữ liệu và khả năng lưu trữ các thành phần như Test Case, Test Suite, Keywords và Data Files theo định dạng phù hợp với quy trình phát triển phần mềm hiện đại.
- Nghiên cứu khả năng quản lý môi trường kiểm thử thông qua hệ thống Profile, biến môi trường và tham số cấu hình, từ đó đánh giá mức độ linh hoạt khi triển khai kiểm thử trên nhiều môi trường khác nhau như Development, Staging và Production.
- Đánh giá chức năng kiểm thử API, bao gồm khả năng cấu hình HTTP Request, quản lý Authentication, xử lý dữ liệu đầu vào và xây dựng các Assertion nhằm xác minh tính chính xác của kết quả trả về từ hệ thống.
- Phân tích cơ chế thiết kế, quản lý và thực thi các kịch bản kiểm thử. Nội dung này tập trung vào khả năng hỗ trợ kiểm thử trên nhiều nền tảng khác nhau như Web,

Desktop, Mobile và API, đồng thời đánh giá tính mở rộng của Test Runner và Automation Engine trong quá trình thực thi.

- Nghiên cứu quy trình phân tích mã nguồn và tự động sinh kịch bản kiểm thử. Hệ thống sử dụng kỹ thuật lập chỉ mục mã nguồn dựa trên AST và Code Analysis để xây dựng mô hình biểu diễn dự án, từ đó hỗ trợ AI Agent nhận diện luồng nghiệp vụ, API Endpoint, giao diện người dùng và đề xuất các Test Case phù hợp.
- Đánh giá các chức năng phân tích và báo cáo, bao gồm lịch sử thực thi, thống kê kết quả kiểm thử, quản lý bằng chứng kiểm thử, trực quan hóa cấu trúc dự án thông qua Knowledge Graph, Coverage Analysis và các biểu đồ hỗ trợ theo dõi chất lượng phần mềm.
- Phân tích các rủi ro kỹ thuật liên quan đến kiến trúc hệ thống, cơ chế IPC, quản lý dữ liệu, tích hợp Appium, AI Agent, dịch vụ bên ngoài và khả năng duy trì hệ thống trong quá trình phát triển lâu dài.
- Đề xuất các giải pháp nhằm nâng cao chất lượng sản phẩm, cải thiện hiệu quả kiểm thử tự động, tăng khả năng bảo trì, mở rộng và hỗ trợ quy trình phát hành phần mềm trong môi trường thực tế.

### 1.2.3 Phạm vi không bao gồm

Do giới hạn về thời gian, nguồn lực và phạm vi nghiên cứu, đề tài không tập trung vào một số nội dung ngoài phạm vi đánh giá chính của hệ thống. Cụ thể:

- Không thực hiện kiểm thử đầy đủ trên tất cả hệ điều hành, thiết bị và môi trường triển khai khác nhau.
- Không tiến hành đo kiểm hiệu năng chuyên sâu bằng các công cụ benchmark hoặc stress testing chuyên dụng.
- Không thực hiện kiểm thử bảo mật ở mức độ chuyên gia như Penetration Testing, Vulnerability Assessment hoặc Red Teaming.
- Không đánh giá các yếu tố liên quan đến hạ tầng triển khai thực tế như cân bằng tải, khả năng chịu lỗi, kiến trúc phân tán hoặc khả năng mở rộng ở quy mô lớn.
- Không kiểm chứng toàn bộ các chức năng được mô tả trong tài liệu bằng thực nghiệm; một số nội dung được đánh giá dựa trên tài liệu thiết kế, mã nguồn và hiện trạng phát triển của hệ thống.
- Không nghiên cứu chi tiết thuật toán, mô hình học máy hoặc cơ chế huấn luyện của các dịch vụ trí tuệ nhân tạo được tích hợp trong hệ thống.

Do đó, các kết quả của đề tài chủ yếu tập trung vào việc phân tích kiến trúc, đánh giá chất lượng thiết kế, xây dựng định hướng kiểm thử và nhận diện rủi ro kỹ thuật nhằm hỗ trợ quá trình hoàn thiện hệ thống trong các giai đoạn phát triển tiếp theo.

### 1.3 Từ điển thuật ngữ

| STT | Thuật ngữ              | Định nghĩa                                                                                                                                                                                                      |
|-----|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | Use Case               | Biểu đồ mô tả những yêu cầu của hệ thống.                                                                                                                                                                       |
| 2   | Tester                 | Là người thực hiện kiểm thử, ghi nhận kết quả , báo cáo lỗi và kiểm tra lại lỗi                                                                                                                                 |
| 3   | Test Case              | Kịch bản kiểm thử. Là một tập hợp các bước (steps), điều kiện đầu vào và kết quả mong đợi được định nghĩa để xác minh một chức năng cụ thể của phần mềm có hoạt động đúng hay không.                            |
| 4   | Test Suite             | Bộ kiểm thử. Là tập hợp gồm nhiều Test Case được nhóm lại với nhau để thực thi theo một trình tự hoặc một cấu hình môi trường nhất định.                                                                        |
| 5   | Keyword-Driven Testing | Phương pháp kiểm thử hướng từ khóa. Phương pháp thiết kế kịch bản kiểm thử bằng cách sử dụng các "từ khóa" đại diện cho hành động (VD: click, type-text, assert-text) thay vì phải viết mã lập trình trực tiếp. |
| 6   | Object Repository      | Kho lưu trữ đối tượng. Nơi quản lý tập trung các bộ định vị (Locators/Selectors) của các phần tử giao diện (UI Elements) dùng trong dự án kiểm thử, giúp tái sử dụng và dễ dàng bảo trì.                        |
| 7   | Environment Profile    | Hồ sơ môi trường. Cấu hình chứa các biến số (Variables) và tham số đặc thù được sử dụng cho một                                                                                                                 |

|    |                                   |                                                                                                                                                                                                                                                                                                                                              |
|----|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    |                                   | môi trường kiểm thử cụ thể (VD: môi trường Dev, Staging, Production).                                                                                                                                                                                                                                                                        |
| 8  | AI Agent                          | Tác nhân Trí tuệ nhân tạo. Mô hình AI được tích hợp trong Phần mềm để tự động hóa các tác vụ như phân tích mã nguồn, gỡ lỗi (debug) và tương tác với các công cụ kiểm thử trên máy người dùng.                                                                                                                                               |
| 9  | Autogen / Indexer                 | Luồng tính năng tự động sinh kiểm thử. Hệ thống hỗ trợ sao chép (clone) mã nguồn từ URL (GitHub), sau đó Indexer sử dụng kỹ thuật phân tích Cây cú pháp trừu tượng (AST - Abstract Syntax Tree) đa ngôn ngữ (TS, JS, Java, Python...) để lập chỉ mục. Cuối cùng, Autogen sử dụng dữ liệu này để tự động tạo ra các kịch bản kiểm thử API/UI. |
| 10 | IPC (Inter-Process Communication) | Giao tiếp liên tiến trình. Trong kiến trúc ứng dụng Electron, đây là cơ chế an toàn cho phép luồng giao diện người dùng trao đổi dữ liệu với luồng hệ thống lỗi.                                                                                                                                                                             |
| 11 | Monorepo                          | Kiến trúc kho lưu trữ đơn. Phương pháp quản lý mã nguồn trong đó toàn bộ các module, packages và ứng dụng (apps) của hệ thống được lưu trữ chung trong một repository duy nhất (quản lý bằng pnpm workspace, Turborepo).                                                                                                                     |
| 12 | MCP (Model Context Protocol)      | Giao thức ngữ cảnh mô hình. Giao thức tiêu chuẩn được Phần mềm sử dụng để kết nối an toàn các công cụ (Tools), dữ liệu hệ thống (Filesystem) với mô hình ngôn ngữ lớn (LLM) của AI Agent.                                                                                                                                                    |
| 13 | Appium / Maestro                  | Các nền tảng/công cụ thực thi kiểm thử tự động dành riêng cho ứng dụng di động (Mobile App) trên iOS và                                                                                                                                                                                                                                      |

|    |            |                                                                                                                                                                          |
|----|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    |            | Android. Phần mềm đóng vai trò là lớp vỏ tích hợp (Adapter) để điều khiển các công cụ này.                                                                               |
| 14 | Playwright | Framework kiểm thử tự động mã nguồn mở do Microsoft phát triển, được Phần mềm tích hợp dưới dạng Adapter để thực thi các kịch bản kiểm thử trên nền tảng Web và Desktop. |

**Bảng 1.1 – Bảng từ điển thuật ngữ hệ thống JKAuto**

## 1.4 Tổng quát

### 1.4.1 Tổng quát hệ thống

Hệ thống được xây dựng dưới dạng ứng dụng Desktop theo mô hình kiến trúc đa lớp (Multi-tier Architecture), hỗ trợ thiết kế, quản lý và thực thi các kịch bản kiểm thử tự động trên nhiều nền tảng khác nhau như Web, Mobile, Desktop và API. Kiến trúc này giúp tách biệt giữa giao diện người dùng, xử lý nghiệp vụ và các thành phần hỗ trợ thông minh, từ đó nâng cao khả năng mở rộng, bảo trì và tích hợp các công nghệ mới trong tương lai.

Kiến trúc hệ thống gồm ba lớp chính:

- **Lớp giao diện người dùng (Renderer Process):** Được xây dựng bằng React, cung cấp môi trường làm việc trực quan cho người dùng trong việc quản lý dự án, xây dựng Test Case, tổ chức Test Suite, cấu hình môi trường kiểm thử và theo dõi kết quả thực thi. Giao diện được thiết kế theo mô hình Workspace nhằm tập trung toàn bộ hoạt động kiểm thử trong một không gian làm việc thống nhất.
- **Lớp xử lý trung tâm (Main Process & Engine):** Được phát triển bằng Electron và Node.js, chịu trách nhiệm quản lý tệp dự án, giao tiếp liên tiến trình (IPC), quản lý cấu hình hệ thống và điều phối các bộ thực thi kiểm thử. Thành phần này đóng vai trò cầu nối giữa giao diện người dùng với các công cụ tự động hóa bên ngoài như trình duyệt, thiết bị di động và các dịch vụ API.
- **Lớp phân tích và hỗ trợ thông minh (AI & Indexer):** Thực hiện phân tích mã nguồn, lập chỉ mục cấu trúc dự án và hỗ trợ sinh kịch bản kiểm thử tự động. Thành

phần này giúp phát hiện các route, API endpoint, thành phần giao diện và các luồng nghiệp vụ tiềm năng để đề xuất hoặc tạo test case phù hợp, góp phần giảm thời gian xây dựng kiểm thử thủ công.

Toàn bộ dữ liệu dự án được lưu trữ dưới dạng JSON hoặc YAML, bao gồm cấu hình hệ thống, test case, test suite và các tài nguyên kiểm thử khác. Cách tiếp cận này giúp dữ liệu dễ dàng quản lý phiên bản, chia sẻ giữa các thành viên và tích hợp với các hệ thống quản lý mã nguồn như Git.

#### **1.4.2 Cấu trúc tài liệu**

Cấu trúc của tài liệu Đặc tả Yêu cầu Phần mềm (SRS) này được chia thành 3 phần chính:

Phần 1: Giới thiệu chung. Cung cấp cái nhìn tổng quan về nền tảng Phần mềm, bao gồm mục đích, phạm vi sử dụng, bảng giải nghĩa thuật ngữ và mô hình hệ thống tổng quát.

Phần 2: Các yêu cầu chức năng. Phân tích chi tiết các tác nhân tham gia vào hệ thống (Actors), sơ đồ Use-Case tổng quan và phân rã. Đồng thời, đặc tả chi tiết các quy trình nghiệp vụ cốt lõi như: Quản lý không gian làm việc, Soạn thảo kịch bản (Test Case/Test Suite), Tương tác API, Thực thi kiểm thử và Ứng dụng AI (Autogen).

Phần 3: Các yêu cầu phi chức năng. Định nghĩa các ràng buộc về mặt hệ thống bao gồm yêu cầu về giao diện người dùng (UI/UX), tiêu chuẩn bảo mật, hiệu năng xử lý phân luồng và khả năng tương thích trên các hệ điều hành khác nhau.

## **2. Các yêu cầu chức năng**

### **1.5 Các tác nhân**

Hệ thống JKAuto có hai nhóm tác nhân chính (người tương tác trực tiếp) và ba tác nhân phụ/ngoài (hệ thống hoặc dịch vụ phản hồi). Bảng dưới đây tổng hợp toàn bộ tác nhân, phân loại vai trò và use-case liên quan:

| <b>Tên tác nhân</b>                         | <b>Loại</b>    | <b>Mô tả vai trò</b>                                                                                                                                                                                                    |
|---------------------------------------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Kỹ sư QA/QC                                 | Tác nhân chính | Người dùng cuối chính. Tạo và quản lý dự án, soạn thảo Test Case theo phương pháp Keyword-Driven, thực thi kiểm thử, theo dõi báo cáo kết quả và quản lý dữ liệu kiểm thử (Data File).                                  |
| Kỹ sư Kiểm thử Tự động hóa / Lập trình viên | Tác nhân chính | Người dùng có kỹ năng kỹ thuật cao. Xây dựng Custom Keyword, viết API Test, quản lý Object Repository, cấu hình Environment Profile, tích hợp AI Agent và thiết lập kiểm thử Mobile qua Appium.                         |
| Hệ thống (Engine / File System)             | Tác nhân phụ   | Thực thể nội bộ của JKAuto. Bao gồm Automation Engine (Playwright/Appium Adapter), IPC Bridge (Electron main process) và File System. Tự động phản hồi theo yêu cầu từ người dùng.                                      |
| AI Agent (LLM Service)                      | Tác nhân phụ   | Dịch vụ trí tuệ nhân tạo (tương thích OpenAI API). Nhận ngữ cảnh dự án qua MCP Tools, phân tích yêu cầu bằng ngôn ngữ tự nhiên, sinh danh sách Test Step, đề xuất sửa lỗi và tương tác với hệ thống file qua edit mode. |
| Appium Server / Thiết bị Mobile             | Tác nhân ngoài | Máy chủ Appium cục bộ và thiết bị Android/iOS (thật hoặc giả lập). Cung cấp WebDriver session, stream màn hình (screpy/MJPEG) và xử lý lệnh tap/swipe từ JKAuto.                                                        |



| <b>Tên tác nhân</b>    | <b>Loại</b>    | <b>Mô tả vai trò</b>                                                                                                                                                            |
|------------------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Máy chủ API đích (SUT) | Tác nhân ngoài | Hệ thống phần mềm đang được kiểm thử (System Under Test). Tiếp nhận HTTP request từ API Adapter hoặc Playwright, trả về response để Engine so sánh với Assertion đã định nghĩa. |

Bảng 1.1 – Bảng tổng hợp các tác nhân của hệ thống JKAuto

## 1.6 Các chức năng của hệ thống

Với định hướng là một nền tảng kiểm thử đa năng, hệ thống Phần mềm cung cấp một hệ sinh thái các chức năng toàn diện, cho phép người dùng quản lý, thiết kế và thực thi kiểm thử trên nhiều nền tảng khác nhau (Web, Desktop, Mobile, API). Các chức năng cốt lõi của hệ thống bao gồm:

Quản lý Không gian làm việc và Dự án (Workspace & Project Management):

- Cho phép người dùng khởi tạo, lưu trữ và tổ chức cấu trúc thư mục của các dự án kiểm thử một cách khoa học.
- Quản lý cấu hình Môi trường (Environment Profiles) linh hoạt, hỗ trợ định nghĩa và phân giải (resolve) các biến số toàn cục cho từng môi trường (Dev, Staging, Production).

Thiết kế Kịch bản và Bộ kiểm thử (Test Case & Test Suite Authoring):

- Cung cấp giao diện soạn thảo trực quan dựa trên phương pháp hướng từ khóa (Keyword-driven), cho phép thêm, xóa, sắp xếp các bước kiểm thử (Steps) thông qua thao tác kéo thả.
- Hỗ trợ đóng gói các kịch bản kiểm thử (Test Cases) thành các Bộ kiểm thử (Test Suites) để thực thi hàng loạt với các chính sách linh hoạt (như continueOnFailure).

- Quản lý Kho lưu trữ đối tượng (Object Repository) tập trung, lưu trữ đa dạng các bộ định vị (Locators) của phần tử giao diện.

#### Tương tác và Kiểm thử API (API Testing):

- Cung cấp công cụ giả lập các yêu cầu HTTP (GET, POST, PUT, DELETE, v.v.), hỗ trợ cấu hình Header, Body, và các phương thức xác thực (Authentication).
- Hỗ trợ nhập (Import) cấu trúc từ cURL hoặc OpenAPI/Swagger và thiết lập các điều kiện xác nhận (Assertions) để tự động kiểm tra dữ liệu trả về.

#### Tích hợp Kiểm thử Thiết bị di động (Mobile Testing Integration):

- Quản lý kết nối với các thiết bị di động thật hoặc máy ảo (Android/iOS) thông qua các bộ chuyển đổi Appium và Maestro.
- Hỗ trợ phản chiếu màn hình và trình theo dõi phần tử để thu thập bộ định vị ngay trên ứng dụng.

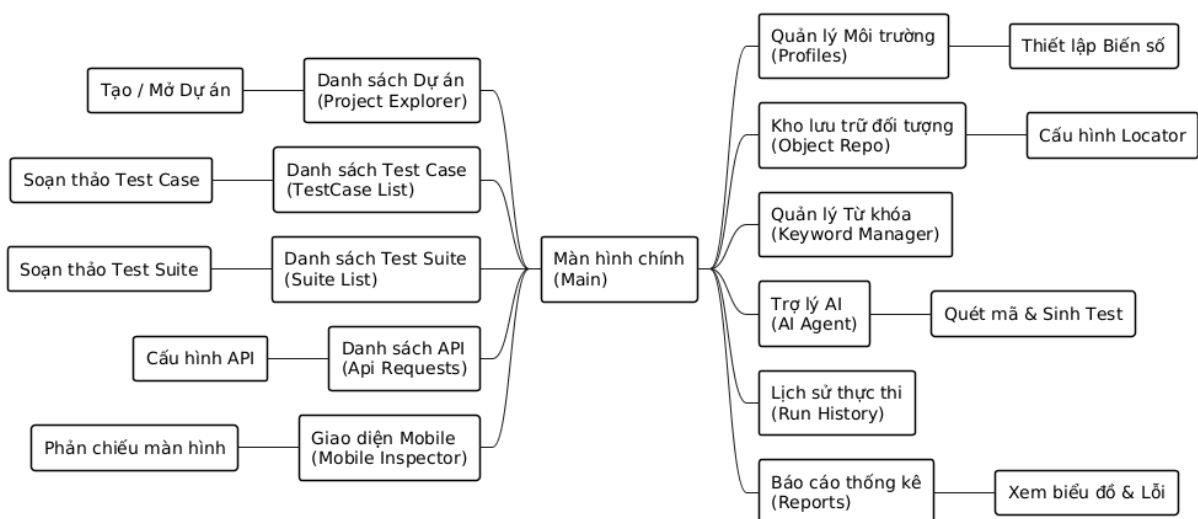
#### Tự động hóa bằng Trí tuệ nhân tạo (AI Autogen & Analysis):

- Tích hợp công cụ quét và lập chỉ mục (Repository Indexer) để tự động đọc hiểu kiến trúc mã nguồn của dự án mục tiêu.
- Sử dụng AI Agent để phân tích, đề xuất từ khóa, gỡ lỗi (debug) và tự động sinh ra các tệp kịch bản kiểm thử chuẩn định dạng JSON/YAML.

#### Thực thi kiểm thử và Báo cáo

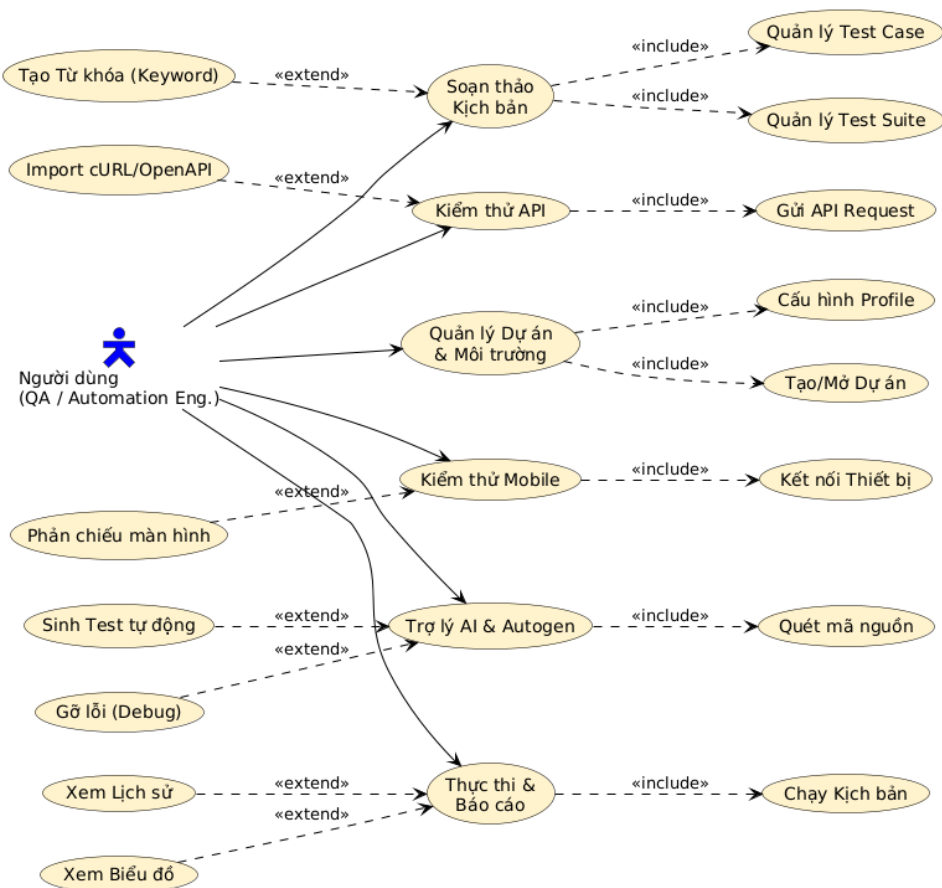
- Trình thực thi cốt lõi có khả năng phân luồng tự động và gọi các công cụ kiểm thử bên dưới (Playwright, Appium) để thao tác trực tiếp lên phần mềm mục tiêu.
- Thu thập log sự kiện theo thời gian thực, lưu trữ lịch sử thực thi và kết xuất các báo cáo trực quan bằng biểu đồ thống kê tỷ lệ Pass/Fail kèm ảnh chụp màn hình lỗi.

## 1.7 Sơ đồ chức năng



Hình 2.1 – Sơ đồ chức năng của hệ thống JKAuto

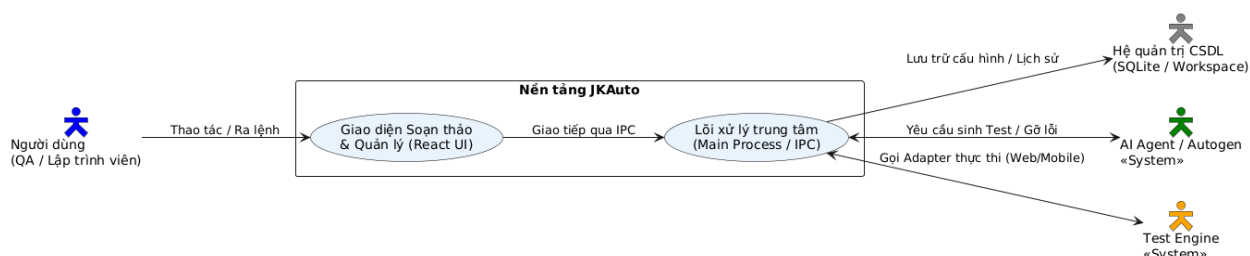
## 1.8 Biểu đồ use – case tổng quan.



Hình 1.1 – Biểu đồ Use-Case tổng quan tương tác giữa các tác nhân và hệ thống Phần mềm

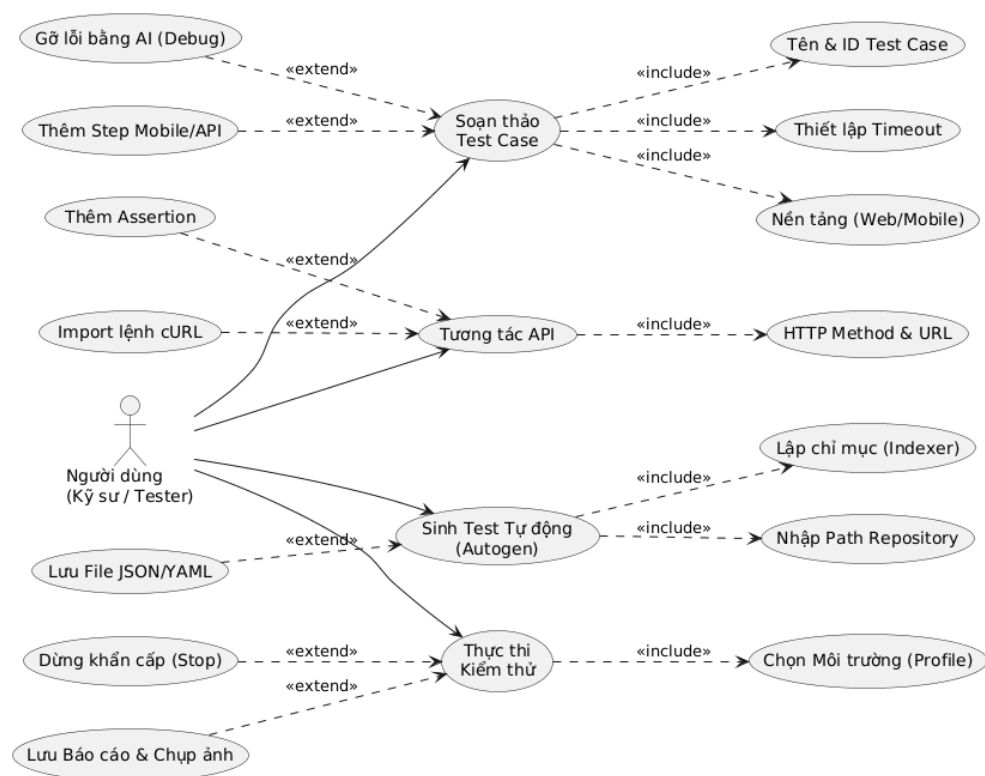
## 1.9 Biểu đồ use – case phân rã

### 1.9.1 Sự tương tác của các tác nhân với hệ thống



Hình 1.2 – Biểu đồ thể hiện sự tương tác của các tác nhân với hệ thống Phần mềm

### 1.9.2 Các use – case đối với người dùng



Hình 1.3 – Biểu đồ Use-Case phân rã các luồng nghiệp vụ đối với người dùng

### 1.10 Quy trình nghiệp vụ

Để có thể thực hiện một chu trình kiểm thử tự động hoàn chỉnh trên nền tảng Phần mềm, người dùng cần trải qua một quy trình nghiệp vụ logic và xuyên suốt từ khâu thiết lập đến khâu đánh giá. Quy trình này được chia thành các bước cơ bản như sau:

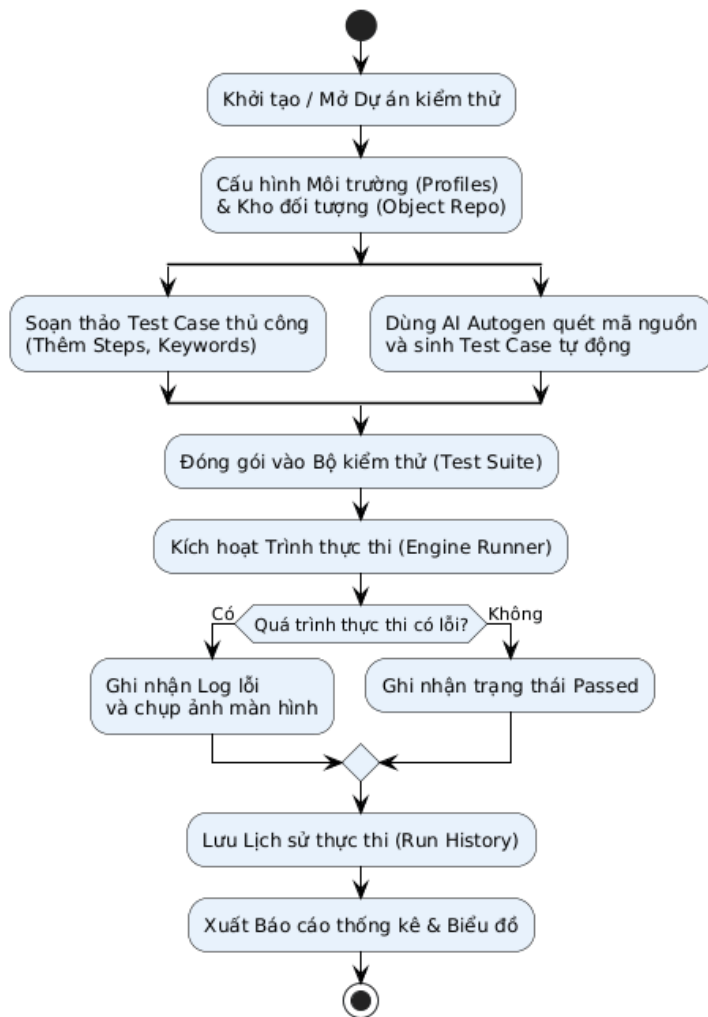
Bước 1: Khởi tạo và Thiết lập: Ban đầu, kỹ sư kiểm thử cần tạo mới hoặc mở một dự án (Project) đã có trên hệ thống. Sau đó, người dùng tiến hành thiết lập các hồ sơ môi trường (Environment Profiles) và khai báo các bộ định vị phần tử vào Kho lưu trữ đối tượng (Object Repository).

Bước 2: Thiết kế kịch bản (Authoring): Người dùng tiến hành tạo các Kịch bản kiểm thử (Test Case). Tại bước này, người dùng có hai lựa chọn:

- Soạn thảo thủ công: Kéo thả và sử dụng các Từ khóa (Keywords) có sẵn để tạo từng bước (Step) tương tác với Web, Mobile hoặc API.
- Sử dụng AI (Autogen): Cung cấp đường dẫn mã nguồn (Repository) để Trợ lý AI tự động quét, lập chỉ mục và sinh ra các Kịch bản kiểm thử.

Bước 3: Thực thi kiểm thử: Sau khi kịch bản hoàn thiện, người dùng chọn môi trường chạy và kích hoạt Test Engine. Hệ thống sẽ tự động gọi các bộ chuyển đổi (Playwright, Appium, Maestro) để thao tác trực tiếp lên phần mềm mục tiêu.

Bước 4: Giám sát và Đánh giá: Trong quá trình chạy, người dùng theo dõi log sự kiện theo thời gian thực. Khi kết thúc, người dùng truy cập mục Báo cáo (Reports) để xem lịch sử chạy, phân tích các biểu đồ thống kê và kiểm tra hình ảnh chụp màn hình của các bước kiểm thử bị lỗi (nếu có).



Hình 1.4 – Sơ đồ hoạt động quy trình nghiệp vụ kiểm thử tự động trên nền tảng Phần mềm

## 1.11 Đặc tả use – case

### 1.11.1 Khởi tạo / Mở dự án (Workspace)

|                       |                                                                                                            |
|-----------------------|------------------------------------------------------------------------------------------------------------|
| <b>Mô tả</b>          | Người dùng tạo một không gian làm việc mới hoặc mở một dự án kiểm thử đã có sẵn để bắt đầu phiên làm việc. |
| <b>Các tác nhân</b>   | Người dùng (Kỹ sư QA/QC, Lập trình viên), Hệ thống                                                         |
| <b>Tiền điều kiện</b> | Ứng dụng JKAuto đã được cài đặt và khởi chạy thành công trên máy tính người dùng.                          |

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Hậu điều kiện</b>     | <p>Giao diện làm việc (Workspace) của dự án được tải thành công, hiển thị đầy đủ cây thư mục dự án và sẵn sàng cho các thao tác tiếp theo.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Các luồng sự kiện</b> | <p>Luồng cơ bản:</p> <ol style="list-style-type: none"> <li>1. Người dùng mở ứng dụng JKAuto.</li> <li>2. Hệ thống hiển thị màn hình Welcome với các tùy chọn: "Tạo dự án mới" hoặc "Mở dự án".</li> <li>3. Người dùng chọn "Tạo dự án mới", nhập tên dự án, chọn loại dự án (Web/Mobile/Desktop/API) và chỉ định thư mục lưu trữ cục bộ.</li> <li>4. Hệ thống tạo cấu trúc thư mục chuẩn (test-cases/, profiles/, keywords/, data-files/, object-repository/, test-suites/) và file project.json.</li> <li>5. Hệ thống tải cấu hình dự án, hiển thị cây thư mục bên trái (Explorer) và chuyển người dùng sang màn hình làm việc chính.</li> </ol> <p>Luồng ngoại lệ:</p> <ol style="list-style-type: none"> <li>1. Thư mục được chọn không có quyền ghi (Read-only): Hệ thống hiển thị thông báo lỗi "Không có quyền truy cập thư mục" và yêu cầu chọn lại.</li> <li>2. Tệp project.json bị hỏng hoặc sai định dạng: Hệ thống hiển thị thông báo "Dự án không hợp lệ hoặc tệp tin bị hỏng".</li> </ol> |

|  |                                                                                                    |
|--|----------------------------------------------------------------------------------------------------|
|  | 3. Trùng tên dự án trong workspace: Hệ thống tự động đổi tên với hậu tố " Copy" để tránh xung đột. |
|--|----------------------------------------------------------------------------------------------------|

Bảng 1.2 – Bảng đặc tả use-case Khởi tạo / Mở dự án (Workspace)

| STT | Trường dữ liệu       | Kiểu dữ liệu   | Bắt buộc | Điều kiện hợp lệ                                              | Ví dụ                       |
|-----|----------------------|----------------|----------|---------------------------------------------------------------|-----------------------------|
| 1   | Tên dự án            | Text           | Có       | Dài 3–50 ký tự, không chứa ký tự đặc biệt như \ / : * ? " < > | JKAuto_Smoke_Test           |
| 2   | Loại dự án (Type)    | Dropdown       | Có       | Chọn một trong: Web, Mobile, Desktop, API                     | Web                         |
| 3   | Đường dẫn lưu (Path) | Directory Path | Có       | Phải là thư mục tồn tại và có quyền ghi                       | C:\Projects                 |
| 4   | Mô tả dự án          | Textarea       | Không    | Tối đa 255 ký tự                                              | Dự án kiểm thử web bán hàng |

Bảng 1.3 – Dữ liệu chức năng Khởi tạo / Mở dự án (Workspace)



### 1.11.2 Tạo và soạn thảo Test Case

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả</b>             | Người dùng tạo mới một kịch bản kiểm thử và cấu hình các bước (Steps) theo phương pháp hướng từ khóa (Keyword-Driven) thông qua giao diện bảng trực quan, có hỗ trợ kéo thả, sao chép, undo/redo và kiểm thử theo nền tảng.                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Các tác nhân</b>      | Người dùng, Hệ thống (Engine, File System)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Tiền điều kiện</b>    | Người dùng đã mở một dự án thành công và đang ở trong màn hình làm việc chính.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Hậu điều kiện</b>     | Test Case được lưu thành công dưới dạng tệp .test.json hoặc .test.yaml trong thư mục test-cases/ của dự án.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Các luồng sự kiện</b> | <p>Luồng cơ bản:</p> <ol style="list-style-type: none"><li>1. Người dùng nhấp chuột phải vào thư mục test-cases/ trong cây thư mục và chọn "Tạo Test Case mới".</li><li>2. Hệ thống hiển thị hộp thoại yêu cầu nhập tên tệp.</li><li>3. Người dùng nhập tên và xác nhận. Hệ thống mở tab soạn thảo với bảng bước rỗng.</li><li>4. Người dùng chọn nền tảng (Platform: web/mobile/desktop/api) trên thanh công cụ. Hệ thống lọc và hiển thị danh sách từ khóa phù hợp với nền tảng.</li><li>5. Người dùng nhấn "Thêm bước" (Add Step) hoặc phím Alt+A để thêm dòng mới. Hệ thống thêm một bước mặc định với từ khóa "click".</li></ol> |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>6. Người dùng chọn từ khóa (ví dụ: navigate-to, type-text, assert-text) từ dropdown, điền Object Ref (selector), Input và Expected.</p> <p>7. Người dùng kéo thả các dòng để sắp xếp thứ tự, hoặc sao chép/dán bước qua menu chuột phải.</p> <p>8. Người dùng nhấn Ctrl+S (hoặc nút Save) để lưu. Hệ thống ghi tệp xuống ổ đĩa và cập nhật cây thư mục.</p> <p>Luồng ngoại lệ:</p> <p>1. Người dùng nhập tên tệp trùng lặp: Hệ thống thông báo và yêu cầu chọn tên khác.</p> <p>2. Người dùng lưu khi có bước bị thiếu dữ liệu bắt buộc: Hệ thống vẫn cho phép lưu nhưng highlight bước lỗi khi chạy.</p> <p>3. Tệp bị sửa từ bên ngoài trong khi tab đang mở: Hệ thống phát hiện thay đổi và đề xuất reload tệp.</p> |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Bảng 1.4 – Bảng đặc tả use-case Tạo và soạn thảo Test Case

| STT | Trường dữ liệu    | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                    | Ví dụ                   |
|-----|-------------------|--------------|----------|-------------------------------------|-------------------------|
| 1   | Tên tệp Test Case | Text         | Có       | Không trùng lặp trong cùng thư mục; | login_success.test.yaml |

| STT | Trường dữ liệu      | Kiểu dữ liệu  | Bắt buộc | Điều kiện hợp lệ                                              | Ví dụ       |
|-----|---------------------|---------------|----------|---------------------------------------------------------------|-------------|
|     |                     |               |          | không chứa khoảng trắng đầu/cuối                              |             |
| 2   | Nền tảng (Platform) | Dropdown      | Không    | inherit   web   mobile   desktop   api; mặc định là inherit   | web         |
| 3   | Thiết bị (Device)   | Text/Datalist | Không    | Chỉ hiển thị khi Platform = mobile; ví dụ: iPhone 14, Pixel 7 | iPhone 14   |
| 4   | Từ khóa (Keyword)   | Dropdown      | Có       | Lấy từ danh sách engine, lọc theo nền tảng                    | assert-text |
| 5   | Object Ref          | Text          | Tùy KW   | CSS selector, XPath, accessibility                            | #login-btn  |

| STT | Trường dữ liệu      | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                              | Ví dụ |
|-----|---------------------|--------------|----------|-----------------------------------------------|-------|
|     |                     |              |          | ID hoặc tên từ object-repo                    |       |
| 6   | Timeout (ms)        | Number       | Không    | Số nguyên dương; null = mặc định 30 000 ms    | 10000 |
| 7   | Continue on Failure | Checkbox     | Không    | true/false; khi true bước lỗi không dừng test | false |

Bảng 1.5 – Dữ liệu chức năng Tạo và soạn thảo Test Case

### 1.11.3 Thực thi kiểm thử (Execute Test Case)

|                       |                                                                                                                                                 |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả</b>          | Người dùng kích hoạt Engine để chạy một Test Case đã soạn thảo, theo dõi kết quả từng bước theo thời gian thực và lưu lịch sử sau khi hoàn tất. |
| <b>Các tác nhân</b>   | Người dùng, Engine (Playwright / Appium / API Adapter), Hệ thống                                                                                |
| <b>Tiền điều kiện</b> | Có ít nhất một Test Case hợp lệ trong dự án.                                                                                                    |

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                          | <p>Máy tính đã cài đặt môi trường phù hợp (Node.js, Playwright browsers, Appium nếu chạy mobile).</p> <p>Đã cấu hình ít nhất một Environment Profile (hoặc dùng mặc định).</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Hậu điều kiện</b>     | <p>Quá trình chạy hoàn tất; kết quả từng bước (Pass/Fail) được hiển thị trực tiếp trong bảng bước và lưu vào thư mục .autotest/runs/.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Các luồng sự kiện</b> | <p>Luồng cơ bản:</p> <ol style="list-style-type: none"> <li>1. Người dùng mở Test Case và nhấn nút "Run" (F5) hoặc chọn Run từ thanh công cụ.</li> <li>2. Hệ thống tự động lưu tệp Test Case (auto-save) trước khi chạy.</li> <li>3. Hệ thống đọc Environment Profile đang kích hoạt, nạp biến môi trường vào runner.</li> <li>4. Engine khởi động Adapter tương ứng (Playwright cho web/desktop, Appium cho mobile, API Adapter cho api).</li> <li>5. Engine thực thi tuần tự từng bước; mỗi bước phát sự kiện ENGINE_STEP_EVENT cập nhật trạng thái (running/passed/failed) trực tiếp lên bảng.</li> <li>6. Khi hoàn tất, Engine phát ENGINE_RUN_COMPLETE; hệ thống lưu bản ghi lịch sử và hiển thị tổng kết (Passed/Failed, thời gian).</li> </ol> |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>Luồng ngoại lệ:</p> <ol style="list-style-type: none"> <li>1. Không tìm thấy trình duyệt Playwright (browsers chưa cài): Hệ thống hiển thị banner hướng dẫn cài đặt.</li> <li>2. Adapter bị lỗi kết nối (Appium không chạy, thiết bị ngắt kết nối): Engine hủy ngay, báo "Lỗi kết nối Adapter".</li> <li>3. Một bước bị lỗi và Continue on Failure = false: Engine dừng tại bước đó, đánh dấu các bước còn lại là "skipped".</li> <li>4. Người dùng nhấn Stop (Shift+F5) giữa chừng: Engine nhận lệnh hủy, trạng thái run = "stopped".</li> </ol> |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Bảng 1.6 – Bảng đặc tả use-case Thực thi kiểm thử (Execute Test Case)

| STT | Trường dữ liệu      | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                                          | Ví dụ   |
|-----|---------------------|--------------|----------|-----------------------------------------------------------|---------|
| 1   | Environment Profile | Dropdown     | Có       | Chọn một profile đã định nghĩa; nếu không có dùng Default | staging |
| 2   | Headless Mode       | Checkbox     | Không    | true = chạy ngầm không mở giao diện trình duyệt           | false   |
| 3   | Debug Mode          | Checkbox     | Không    | true = dừng sau mỗi bước, chờ lệnh                        | false   |

| STT | Trường dữ liệu      | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                       | Ví dụ |
|-----|---------------------|--------------|----------|----------------------------------------|-------|
|     |                     |              |          | Next (F6) từ người dùng                |       |
| 4   | Continue on Failure | Checkbox     | Không    | true = tiếp tục chạy dù bước trước lỗi | false |

Bảng 1.7 – Dữ liệu chức năng Thực thi kiểm thử (Execute Test Case)

#### 1.11.4 Gỡ lỗi kiểm thử theo từng bước (Debug Mode)

|                          |                                                                                                                                                                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả</b>             | Người dùng chạy Test Case ở chế độ Debug để tạm dừng sau mỗi bước, quan sát trạng thái ứng dụng đích và tiến hành gỡ lỗi tương tác từng bước một.                                                                                             |
| <b>Các tác nhân</b>      | Người dùng, Engine, Hệ thống                                                                                                                                                                                                                  |
| <b>Tiền điều kiện</b>    | Test Case đã được lưu, Engine và môi trường đã sẵn sàng.                                                                                                                                                                                      |
| <b>Hậu điều kiện</b>     | Người dùng xác định được nguyên nhân lỗi; có thể dừng sớm (Stop) hoặc tiếp tục đến hết kịch bản.                                                                                                                                              |
| <b>Các luồng sự kiện</b> | <p>Luồng cơ bản:</p> <ol style="list-style-type: none"> <li>1. Người dùng nhấn nút "Debug" (F6) trên thanh công cụ Test Case.</li> <li>2. Engine khởi động Adapter, thực thi bước đầu tiên, sau đó DỪNG và chờ lệnh từ người dùng.</li> </ol> |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>3. Giao diện hiển thị trạng thái "Paused at step N", highlight dòng bước đang dừng.</p> <p>4. Người dùng quan sát ứng dụng đích (trình duyệt, thiết bị), kiểm tra selector và biến.</p> <p>5. Người dùng nhấn "Next Step" (F6) để tiếp tục chạy bước kế tiếp.</p> <p>6. Lặp lại bước 2–5 cho đến khi hoàn tất hoặc người dùng nhấn Stop.</p> <p>Luồng ngoại lệ:</p> <p>1. Bước hiện tại bị lỗi (element không tìm thấy, assertion sai): Hệ thống hiển thị thông báo lỗi chi tiết và cho phép người dùng tiếp tục hoặc dừng.</p> <p>2. Timeout chờ lệnh Next quá lâu: Người dùng có thể nhấn Stop để hủy session debug.</p> |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Bảng 1.8 – Bảng đặc tả use-case Gỡ lỗi kiểm thử theo từng bước (Debug Mode)

| STT | Trường dữ liệu | Kiểu dữ liệu  | Bắt buộc | Điều kiện hợp lệ                               | Ví dụ |
|-----|----------------|---------------|----------|------------------------------------------------|-------|
| 1   | Chế độ Debug   | Toggle/Button | Có       | Kích hoạt bằng nút Debug (F6) thay vì Run (F5) | true  |



| STT | Trường dữ liệu | Kiểu dữ liệu    | Bắt buộc            | Điều kiện hợp lệ                                 | Ví dụ    |
|-----|----------------|-----------------|---------------------|--------------------------------------------------|----------|
| 2   | Lệnh Next Step | Button/Shortcut | Có (khi đang debug) | Chỉ khả dụng khi Engine đang ở trạng thái Paused | F6       |
| 3   | Lệnh Stop      | Button/Shortcut | Không               | Hủy toàn bộ session debug, giải phóng Adapter    | Shift+F5 |

Bảng 1.9 – Dữ liệu chức năng Gỡ lỗi kiểm thử theo từng bước (Debug Mode)

#### 1.11.5 Quản lý và thực thi Test Suite

|                          |                                                                                                                                                   |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả</b>             | Người dùng tạo một Test Suite để nhóm nhiều Test Case lại, cấu hình thứ tự thực thi, gán Environment Profile riêng và chạy toàn bộ suite một lần. |
| <b>Các tác nhân</b>      | Người dùng, Engine, Hệ thống                                                                                                                      |
| <b>Tiền điều kiện</b>    | Dự án có ít nhất một Test Case.                                                                                                                   |
| <b>Hậu điều kiện</b>     | Suite được lưu dưới dạng .suite.json; sau khi chạy, kết quả từng Test Case được hiển thị và lưu vào lịch sử.                                      |
| <b>Các luồng sự kiện</b> | Luồng cơ bản:                                                                                                                                     |

1. Người dùng nhấp chuột phải vào thư mục test-suites/ và chọn "Tạo Test Suite mới".
2. Hệ thống tạo tệp .suite.json và mở tab SuiteEditor.
3. Người dùng đặt tên Suite, chọn Environment Profile và bật/tắt tùy chọn "Continue on Failure".
4. Người dùng chọn Test Case từ dropdown "Thêm Test Case" và nhấn "Add Case" để bổ sung vào danh sách.
5. Người dùng kéo thả hoặc dùng nút ↑/↓ để sắp xếp lại thứ tự; tick/bỏ tick ô "Enabled" để bật/tắt từng Test Case.
6. Người dùng nhấn "Run Suite" để chạy. Engine thực thi từng Test Case theo thứ tự, phát sự kiện ENGINE\_SUITE\_EVENT cập nhật trạng thái từng dòng theo thời gian thực.
7. Khi hoàn tất, footer hiển thị tổng kết: "N/M cases passed (Xs)".

Luồng ngoại lệ:

1. Một Test Case không tìm thấy tệp (đã xóa hoặc đổi đường dẫn): Engine bỏ qua Test Case đó và ghi nhận lỗi "File not found".
2. Người dùng nhấn Stop giữa chừng: Engine hoàn tất Test Case hiện tại rồi dừng, các Case còn lại chuyển sang "skipped".

Bảng 1.10 – Bảng đặc tả use-case Quản lý và thực thi Test Suite

| STT | Trường dữ liệu      | Kiểu dữ liệu      | Bắt buộc | Điều kiện hợp lệ                                      | Ví dụ                   |
|-----|---------------------|-------------------|----------|-------------------------------------------------------|-------------------------|
| 1   | Tên Suite           | Text              | Có       | Không trùng lặp trong cùng thư mục                    | Smoke_Suite             |
| 2   | Environment Profile | Dropdown          | Có       | Profile đã tạo trong dự án; mặc định: default         | staging                 |
| 3   | Continue on Failure | Checkbox          | Không    | true = Suite tiếp tục chạy dù một Case thất bại       | false                   |
| 4   | Test Case (Items)   | Multi-select list | Có       | Ít nhất một Test Case được kích hoạt (enabled = true) | login_success.test.yaml |

| STT | Trường dữ liệu | Kiểu dữ liệu     | Bắt buộc | Điều kiện hợp lệ                              | Ví dụ |
|-----|----------------|------------------|----------|-----------------------------------------------|-------|
| 5   | Thứ tự (Order) | Drag & Drop / ↑↓ | Không    | Số nguyên $\geq 0$ , xác định thứ tự thực thi | 1     |

Bảng 1.11 – Dữ liệu chức năng Quản lý và thực thi Test Suite

#### 1.11.6 Soạn thảo và gửi API Request

|                          |                                                                                                                                                                                                                  |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả</b>             | Người dùng sử dụng giao diện API Request Editor (tương tự Postman) để tạo, gửi và kiểm tra một HTTP request, quản lý authentication, viết assertion và lưu vào lịch sử. Hỗ trợ import từ cURL hoặc OpenAPI spec. |
| <b>Các tác nhân</b>      | Người dùng, Hệ thống (HTTP Handler), Server đích                                                                                                                                                                 |
| <b>Tiền điều kiện</b>    | Người dùng đã mở một dự án. Server đích cần truy cập từ máy local.                                                                                                                                               |
| <b>Hậu điều kiện</b>     | Request được gửi, response hiển thị đầy đủ (status, headers, body, thời gian phản hồi); Assertion được đánh giá tự động; lịch sử lưu tối đa 30 lần gửi gần nhất.                                                 |
| <b>Các luồng sự kiện</b> | Luồng cơ bản:                                                                                                                                                                                                    |

1. Người dùng tạo tệp .request.json trong api-requests/ hoặc import từ OpenAPI/cURL.

2. Hệ thống mở tab RequestEditor với form: Method, URL, Params, Headers, Body, Auth, Assertions.

3. Người dùng nhập Method (GET/POST/PUT/PATCH/DELETE), URL (hỗ trợ biến {{baseUrl}}), cấu hình Auth (Bearer/Basic/API-Key) và Body (JSON/form-urlencoded/raw).

4. Người dùng thêm các Assertion để xác minh status code, header, JSON path hoặc thời gian phản hồi.

5. Người dùng nhấn "Send". Hệ thống (main process) nạp biến môi trường, resolve {{var}}, thực hiện HTTP request.

6. Response trả về hiển thị: Status badge màu, Body (pretty-print JSON), Headers, thời gian. Kết quả Assertion hiển thị Pass/Fail từng dòng.

7. Người dùng có thể click "→ ENV" để lưu giá trị từ body JSON (ví dụ: data.token) vào biến Environment Profile.

Luồng ngoại lệ:

1. URL không hợp lệ hoặc server không phản hồi (timeout): Hệ thống hiển thị lỗi "Network Error" hoặc "Request Timeout".

2. Biến {{baseUrl}} chưa được cấu hình trong Profile: Hệ thống báo lỗi "Base URL not configured".

3. Body JSON không hợp lệ (lỗi cú pháp): Hệ thống highlight lỗi và không gửi request.

Bảng 1.12 – Bảng đặc tả use-case Soạn thảo và gửi API Request

| STT | Trường dữ liệu | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                                           | Ví dụ                 |
|-----|----------------|--------------|----------|------------------------------------------------------------|-----------------------|
| 1   | HTTP Method    | Dropdown     | Có       | GET   POST   PUT   PATCH   DELETE   HEAD   OPTIONS         | POST                  |
| 2   | URL            | Text         | Có       | URL đầy đủ hoặc dùng biến {{baseUrl}}; hỗ trợ query params | {{baseUrl}}/api/login |
| 3   | Auth Type      | Dropdown     | Không    | none   bearer   basic   api-key                            | bearer                |
| 4   | Body Type      | Dropdown     | Không    | none   raw-json   raw-text   form-urlencoded               | raw-json              |

| STT | Trường dữ liệu     | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                                     | Ví dụ  |
|-----|--------------------|--------------|----------|------------------------------------------------------|--------|
| 5   | Assertion Target   | Dropdown     | Không    | status   response-time   header   body-json-path     | status |
| 6   | Assertion Operator | Dropdown     | Không    | eq   ne   contains   not-contains   exists   lt   gt | eq     |

Bảng 1.13 – Dữ liệu chức năng Soạn thảo và gửi API Request

#### 1.11.7 Quản lý Object Repository (Element Locators)

|                       |                                                                                                                                                                                                                                      |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả</b>          | Người dùng tạo và quản lý kho lưu trữ đối tượng UI (.objects.json) – mỗi đối tượng có tên định danh và danh sách locator fallback theo thứ tự ưu tiên – để các Test Step có thể tham chiếu bằng tên thay vì viết selector trực tiếp. |
| <b>Các tác nhân</b>   | Người dùng, Hệ thống (File System)                                                                                                                                                                                                   |
| <b>Tiền điều kiện</b> | Người dùng đã mở một dự án; thư mục object-repository/ tồn tại.                                                                                                                                                                      |

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Hậu điều kiện</b>     | <p>Tập .objects.json được lưu thành công; các Test Step có thể dùng tên đối tượng (objectRef) để engine tự resolve locator phù hợp.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Các luồng sự kiện</b> | <p>Luồng cơ bản:</p> <ol style="list-style-type: none"> <li>1. Người dùng tạo tập .objects.json trong object-repository/ và mở tab ObjectEditor.</li> <li>2. Hệ thống hiển thị danh sách đối tượng dạng accordion (collapsible).</li> <li>3. Người dùng nhấn "Add Object", đặt tên (ví dụ: LoginPage.usernameInput) và mô tả.</li> <li>4. Hệ thống tạo đối tượng với một locator CSS rỗng mặc định.</li> <li>5. Người dùng chọn Strategy (css/xpath/testid/role/label/text) và nhập giá trị locator; đặt Priority (số nhỏ = thử trước).</li> <li>6. Người dùng có thể thêm nhiều locator fallback cho cùng một đối tượng (engine thử theo priority).</li> <li>7. Người dùng nhấn Save (Ctrl+S). Hệ thống ghi tệp và cập nhật autocomplete trong Test Case Editor.</li> </ol> <p>Luồng ngoại lệ:</p> <ol style="list-style-type: none"> <li>1. Người dùng xóa locator cuối cùng của một đối tượng: Hệ thống vô hiệu hóa nút xóa (phải có ít nhất 1 locator).</li> </ol> |



2. Tên đối tượng bị trùng trong cùng file: Hệ thống cảnh báo trùng lặp.

Bảng 1.14 – Bảng đặc tả use-case Quản lý Object Repository (Element Locators)

| ST T | Trường dữ liệu   | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                                             | Ví dụ                   |
|------|------------------|--------------|----------|--------------------------------------------------------------|-------------------------|
| 1    | Tên đối tượng    | Text         | Có       | Unique trong file; nên dùng định dạng PageName.ElementName   | LoginPage.submitBtn     |
| 2    | Mô tả đối tượng  | Text         | Không    | Tối đa 255 ký tự                                             | Nút Đăng nhập trang chủ |
| 3    | Locator Strategy | Dropdown     | Có       | css   xpath   testid   role   label   placeholder   text     | css                     |
| 4    | Locator Value    | Text         | Có       | Giá trị selector tương ứng với strategy đã chọn              | #submit-btn             |
| 5    | Priority         | Number       | Có       | Số nguyên $\geq 0$ ; engine thử locator theo thứ tự tăng dần | 1                       |

Bảng 1.15 – Dữ liệu chức năng Quản lý Object Repository (Element Locators)

### 1.11.8 Quản lý môi trường và Profile biến

|                          |                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả</b>             | Người dùng tạo và quản lý nhiều Environment Profile (dev/staging/production) chứa biến môi trường và cấu hình API riêng, dùng để tham số hóa Test Case và API Request mà không sửa code kịch bản.                                                                                                                                                                                                  |
| <b>Các tác nhân</b>      | Người dùng, Hệ thống                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Tiền điều kiện</b>    | Người dùng đã mở một dự án.                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Hậu điều kiện</b>     | Profile được lưu dưới dạng *.env.json trong thư mục profiles/; các biến {{varName}} được resolve đúng khi chạy test.                                                                                                                                                                                                                                                                               |
| <b>Các luồng sự kiện</b> | <p>Luồng cơ bản:</p> <ol style="list-style-type: none"> <li>1. Người dùng mở EnvManager Dialog (từ menu hoặc status bar).</li> <li>2. Hệ thống liệt kê danh sách Profile hiện có (default, staging...).</li> <li>3. Người dùng tạo Profile mới, đặt tên và nhấn tạo.</li> <li>4. Người dùng nhập các cặp Key=Value (ví dụ: BASE_URL=https://staging.example.com, authToken=Bearer xxx).</li> </ol> |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>5. Người dùng chuyển sang tab "API Config" để cấu hình baseUrl, defaultHeaders và xác thực mặc định cho toàn Profile.</p> <p>6. Người dùng nhấn Save. Hệ thống ghi tệp profiles/&lt;name&gt;.env.json.</p> <p>7. Người dùng chọn Profile làm "Active" từ status bar. Hệ thống áp dụng Profile này cho mọi lần chạy tiếp theo.</p> <p>Luồng ngoại lệ:</p> <p>1. Tên Profile bị trùng: Hệ thống thông báo lỗi và yêu cầu đặt tên khác.</p> <p>2. Xóa Profile đang Active: Hệ thống yêu cầu xác nhận, sau đó tự động chuyển sang Profile "default".</p> |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Bảng 1.16 – Bảng đặc tả use-case Quản lý môi trường và Profile biến

| STT | Trường dữ liệu | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                            | Ví dụ   |
|-----|----------------|--------------|----------|---------------------------------------------|---------|
| 1   | Tên Profile    | Text         | Có       | Unique trong dự án; không chứa khoảng trắng | staging |

| STT | Trường dữ liệu        | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                                          | Ví dụ                      |
|-----|-----------------------|--------------|----------|-----------------------------------------------------------|----------------------------|
| 2   | Biến môi trường (Key) | Text         | Có       | Chữ và số, gạch dưới; không bắt đầu bằng số               | BASE_URL                   |
| 3   | Giá trị (Value)       | Text         | Có       | Chuỗi bất kỳ; dữ liệu nhạy cảm được che giấu trên UI      | https://staging.api.com    |
| 4   | API Base URL          | Text         | Không    | URL gốc áp dụng cho tất cả API Request trong Profile      | https://staging.api.com/v1 |
| 5   | Auth mặc định         | Dropdown     | Không    | none   bearer   basic   api-key – tự động thêm vào header | bearer                     |

Bảng 1.17 – Dữ liệu chức năng Quản lý môi trường và Profile biến

### 1.11.9 Quản lý Custom Keyword

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả</b>             | Người dùng tạo từ khóa tùy chỉnh (Custom Keyword) bằng cách kết hợp nhiều bước cơ bản thành một từ khóa có thể tái sử dụng trong nhiều Test Case khác nhau, giảm trùng lặp và tăng khả năng bảo trì.                                                                                                                                                                                                                                                                          |
| <b>Các tác nhân</b>      | Người dùng, Hệ thống (Engine Keyword Registry)                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Tiền điều kiện</b>    | Người dùng đã mở một dự án; thư mục keywords/ tồn tại.                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Hậu điều kiện</b>     | Tệp .keywords.json được lưu; Custom Keyword xuất hiện trong danh sách từ khóa của Test Case Editor.                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Các luồng sự kiện</b> | <p>Luồng cơ bản:</p> <ol style="list-style-type: none"> <li>1. Người dùng tạo tệp .keywords.json trong thư mục keywords/ và mở tab KeywordEditor.</li> <li>2. Người dùng nhấn "Add Keyword", đặt tên (ví dụ: login_as_admin) và mô tả.</li> <li>3. Người dùng khai báo các tham số đầu vào (Parameters) với tên và kiểu dữ liệu.</li> <li>4. Người dùng thêm các bước nội dung (Steps) tương tự soạn thảo Test Case, sử dụng biến \${param} để tham chiếu tham số.</li> </ol> |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>5. Người dùng lưu (Ctrl+S). Hệ thống cập nhật registry từ khóa; Keyword mới xuất hiện trong dropdown của Test Case Editor.</p> <p>6. Khi Test Case chạy gọi Custom Keyword, Engine mở rộng (expand) các bước con và thực thi tuần tự.</p> <p>Luồng ngoại lệ:</p> <p>1. Tên Keyword trùng với từ khóa hệ thống (built-in): Hệ thống cảnh báo xung đột tên.</p> <p>2. Keyword gọi đệ quy chính nó: Engine phát hiện vòng lặp và báo lỗi "Circular keyword reference".</p> |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Bảng 1.18 – Bảng đặc tả use-case Quản lý Custom Keyword

| STT | Trường dữ liệu | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                                         | Ví dụ                         |
|-----|----------------|--------------|----------|----------------------------------------------------------|-------------------------------|
| 1   | Tên Keyword    | Text         | Có       | Unique, không trùng với built-in keywords, dùng gạch nối | login_as_admin                |
| 2   | Mô tả          | Text         | Không    | Tối đa 255 ký tự, mô tả mục đích từ khóa                 | Đăng nhập với tài khoản admin |

| STT | Trường dữ liệu   | Kiểu dữ liệu   | Bắt buộc | Điều kiện hợp lệ                                                  | Ví dụ                                  |
|-----|------------------|----------------|----------|-------------------------------------------------------------------|----------------------------------------|
| 3   | Tham số (Params) | Key-Value list | Không    | Tên tham số: chữ và số, gạch dưới; kiểu: string/number/boolean    | username: string                       |
| 4   | Các bước (Steps) | Step Table     | Có       | Ít nhất 1 bước; dùng <code>\${param}</code> để tham chiếu tham số | type-text  <br>#user  <br>\${username} |

Bảng 1.19 – Dữ liệu chức năng Quản lý Custom Keyword

#### 1.11.10 Quản lý Data File (Kiểm thử hướng dữ liệu)

|                          |                                                                                                                                                                                                                          |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả</b>             | Người dùng tạo và quản lý tệp dữ liệu kiểm thử (.json/.csv) trong thư mục data-files/, sau đó gắn tệp này vào Test Case để engine tự động lặp lại kịch bản với nhiều bộ dữ liệu đầu vào khác nhau (Data-Driven Testing). |
| <b>Các tác nhân</b>      | Người dùng, Hệ thống                                                                                                                                                                                                     |
| <b>Tiền điều kiện</b>    | Người dùng đã mở một dự án.                                                                                                                                                                                              |
| <b>Hậu điều kiện</b>     | Tệp data file được lưu; Test Case được gắn data binding; khi chạy engine lặp N lần với N bộ dữ liệu, mỗi lần coi như một test run riêng.                                                                                 |
| <b>Các luồng sự kiện</b> | Luồng cơ bản:                                                                                                                                                                                                            |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <ol style="list-style-type: none"> <li>1. Người dùng tạo tệp .json hoặc .csv trong thư mục data-files/.</li> <li>2. Hệ thống mở DataFileEditor dạng bảng cho phép thêm/sửa/xóa dòng và cột.</li> <li>3. Người dùng có thể dùng AI Generator để tự động sinh dữ liệu mẫu từ mô tả.</li> <li>4. Người dùng lưu tệp.</li> <li>5. Trong Test Case Editor, người dùng mở tab "Data Binding" và chọn tệp data file vừa tạo.</li> <li>6. Hệ thống map các cột dữ liệu với các biến {{varName}} trong Test Step.</li> <li>7. Khi chạy, Engine lặp qua từng dòng dữ liệu, thay thế biến và thực thi toàn bộ kịch bản.</li> </ol> <p>Luồng ngoại lệ:</p> <ol style="list-style-type: none"> <li>1. File CSV có cấu trúc không nhất quán (số cột không đồng đều): Hệ thống báo lỗi parse và yêu cầu sửa.</li> <li>2. Tên cột dữ liệu không khớp với biến trong Test Step: Hệ thống cảnh báo "Unresolved variable" nhưng vẫn cho phép chạy.</li> </ol> |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Bảng 2.19 – Bảng đặc tả use-case Quản lý Data File (Kiểm thử hướng dữ liệu)



| STT | Trường dữ liệu | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                                       | Ví dụ              |
|-----|----------------|--------------|----------|--------------------------------------------------------|--------------------|
| 1   | Tên tệp Data   | Text         | Có       | Đuôi .json hoặc .csv; unique trong thư mục data-files/ | login_data.csv     |
| 2   | Tiêu đề cột    | Text         | Có       | Khớp với tên biến {{...}} trong Test Step              | username, password |
| 3   | Dữ liệu dòng   | Mixed        | Có       | Ít nhất 1 dòng dữ liệu; kiểu string/number/boolean     | admin, Admin@123   |

Bảng 1.20 – Dữ liệu chức năng Quản lý Data File (Kiểm thử hướng dữ liệu)

#### 1.11.11 AI Agent – Hỗ trợ sinh và chỉnh sửa Test Case

|                       |                                                                                                                                                                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả</b>          | Người dùng tương tác với AI Agent thông qua giao diện chat tích hợp (AgentPanel) để tự động sinh Test Case từ mô tả ngôn ngữ tự nhiên, gợi ý chỉnh sửa steps, phân tích lỗi chạy test và tự động áp dụng thay đổi vào tệp kịch bản. |
| <b>Các tác nhân</b>   | Người dùng, AI Agent (LLM), MCP Tools (filesystem, jkauto, playwright), Hệ thống                                                                                                                                                    |
| <b>Tiền điều kiện</b> | Đã cấu hình AI Agent (LLM endpoint, API key, model) trong Settings.                                                                                                                                                                 |

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                          | <p>Người dùng đã mở một dự án và có ít nhất một Test Case hoặc mô tả kịch bản.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Hậu điều kiện</b>     | <p>AI Agent trả về phản hồi dưới dạng văn bản và/hoặc khối apply-steps; người dùng có thể nhấn "Apply" để ghi các bước vào Test Case hiện tại.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Các luồng sự kiện</b> | <p>Luồng cơ bản (sinh Test Case):</p> <ol style="list-style-type: none"> <li>1. Người dùng mở AgentPanel (panel phải), chọn Session Mode = "generate-test".</li> <li>2. Người dùng nhập mô tả kịch bản: ví dụ "Tạo test case đăng nhập thành công với tài khoản admin".</li> <li>3. AI Agent phân tích ngữ cảnh (active Test Case, keywords, profiles), gọi MCP tools để đọc file, sinh danh sách steps dưới dạng JSON trong khối <code>````apply-steps [...]````</code>.</li> <li>4. Giao diện hiển thị steps đề xuất kèm nút "Apply". Người dùng xem xét và nhấn Apply.</li> <li>5. Hệ thống ghi đè steps vào tệp Test Case hiện tại và reload editor.</li> </ol> <p>Luồng cơ bản (debug lỗi test):</p> <ol style="list-style-type: none"> <li>1. Người dùng chọn Session Mode = "debug", mô tả lỗi hoặc paste log lỗi vào chat.</li> <li>2. AI Agent đọc nội dung Test Case, phân tích log, đề xuất sửa selector hoặc điều chỉnh assertion.</li> </ol> |

|  |                                                                                                                                                                                                                                                                                                                                                                                                           |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>3. Người dùng Apply gợi ý sửa. Engine chạy lại để xác nhận.</p> <p>Luồng ngoại lệ:</p> <p>1. LLM endpoint không khả dụng hoặc API key sai: Hệ thống báo lỗi "AI Service unavailable".</p> <p>2. AI tạo steps có từ khóa không hợp lệ: Hệ thống validate trước khi Apply và thông báo lỗi schema.</p> <p>3. Edit Mode = "ask" (chỉ đề xuất): Nút Apply hiển thị nhưng không cho ghi file trực tiếp.</p> |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Bảng 1.21 – Bảng đặc tả use-case AI Agent – Hỗ trợ sinh và chỉnh sửa Test Case

| STT | Trường dữ liệu | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                                                     | Ví dụ              |
|-----|----------------|--------------|----------|----------------------------------------------------------------------|--------------------|
| 1   | Session Mode   | Dropdown     | Có       | ask   edit   debug   generate-test                                   | generate-test      |
| 2   | Edit Mode      | Toggle Pill  | Có       | ask (đề xuất)   auto (tự ghi)   auto-with-rollback (tự ghi + backup) | ask                |
| 3   | Nội dung chat  | Textarea     | Có       | Mô tả kịch bản, lỗi hoặc yêu cầu chỉnh sửa; tối đa context window    | Tạo test đăng nhập |

| STT | Trường dữ liệu | Kiểu dữ liệu    | Bắt buộc | Điều kiện hợp lệ                              | Ví dụ                     |
|-----|----------------|-----------------|----------|-----------------------------------------------|---------------------------|
|     |                |                 |          |                                               | thành công                |
| 4   | LLM Model      | Text (Settings) | Có       | Tên model tương thích OpenAI API              | gpt-4o                    |
| 5   | LLM Endpoint   | URL (Settings)  | Có       | URL base của API; mặc định 127.0.0.1:20128/v1 | https://api.openai.com/v1 |

Bảng 1.22 – Dữ liệu chức năng AI Agent – Hỗ trợ sinh và chỉnh sửa Test Case

#### 1.11.12 Kiểm thử ứng dụng Mobile (Appium Inspector)

|                       |                                                                                                                                                                                                                                            |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả</b>          | Người dùng kết nối thiết bị Android/iOS (thật hoặc giả lập), khởi động Appium Server, xem live mirror màn hình thiết bị ngay trong ứng dụng, tap/swipe tương tác để inspect phần tử UI và lấy selector phục vụ soạn thảo Test Case mobile. |
| <b>Các tác nhân</b>   | Người dùng, Hệ thống, Appium Server, Thiết bị Android/iOS (thật hoặc giả lập)                                                                                                                                                              |
| <b>Tiền điều kiện</b> | Appium CLI đã được cài đặt trên máy.<br><br>Android SDK (adb) hoặc Xcode (xcrun simctl) đã được cấu hình.                                                                                                                                  |

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                          | Thiết bị/giả lập đã kết nối và nhận diện được bởi hệ thống.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Hậu điều kiện</b>     | Người dùng xác định được selector của phần tử UI để sử dụng trong Test Case; phiên Appium Session được đóng an toàn.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Các luồng sự kiện</b> | <p>Luồng cơ bản:</p> <ol style="list-style-type: none"> <li>1. Người dùng vào Settings &gt; Appium, cấu hình Host/Port và nhấn "Start Appium Server".</li> <li>2. Hệ thống kiểm tra môi trường (APPIUM_ENV_CHECK): xác nhận appium bin, adb/xcrun khả dụng.</li> <li>3. Người dùng mở AppiumPanel (tab bên phải), chọn thiết bị từ dropdown, nhập Bundle ID / Package Name và nhấn "Connect".</li> <li>4. Hệ thống khởi tạo Appium Session (WDA cho iOS hoặc UIAutomator2 cho Android).</li> <li>5. Màn hình thiết bị được stream trực tiếp vào panel (AndroidMirror dùng screpy H.264, iOS dùng MJPEG WDA).</li> <li>6. Người dùng tap/swipe trực tiếp trên mirror; hệ thống chuyển tọa độ chuẩn hóa [0,1] → Appium W3C pointer actions.</li> <li>7. Người dùng nhấn các nút phần cứng (Home, Back, Volume) từ DeviceToolbar.</li> </ol> |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>8. Người dùng nhấn Stop/Disconnect để đóng session. Hệ thống teardown an toàn khi thoát ứng dụng.</p> <p>Luồng ngoại lệ:</p> <p>1. Appium không cài đặt hoặc không tìm thấy: Banner cảnh báo, nút Start bị vô hiệu hóa.</p> <p>2. Không có thiết bị/giả lập nào kết nối: Dropdown trống, hiển thị hướng dẫn kết nối.</p> <p>3. Session bị ngắt giữa chừng (thiết bị mất kết nối): Hệ thống hiển thị lỗi và cho phép reconnect.</p> |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Bảng 1.23 – Bảng đặc tả use-case Kiểm thử ứng dụng Mobile (Appium Inspector)

| STT | Trường dữ liệu | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                                    | Ví dụ     |
|-----|----------------|--------------|----------|-----------------------------------------------------|-----------|
| 1   | Appium Host    | Text         | Có       | IP hoặc hostname máy chủ Appium; mặc định 127.0.0.1 | 127.0.0.1 |
| 2   | Appium Port    | Number       | Có       | Số nguyên 1024–65535; mặc định 4723                 | 4723      |

| STT | Trường dữ liệu      | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                                              | Ví dụ             |
|-----|---------------------|--------------|----------|---------------------------------------------------------------|-------------------|
| 3   | Thiết bị (Device)   | Dropdown     | Có       | Lấy từ `adb devices` (Android) hoặc `xcrun simctl list` (iOS) | emulator-5554     |
| 4   | Bundle ID / Package | Text         | Có       | Định danh app cần mở (iOS: bundle ID, Android: package name)  | com.example.myapp |
| 5   | Auto Start Appium   | Checkbox     | Không    | true = Appium tự khởi động khi mở dự án có platform=mobile    | false             |

Bảng 1.24 – Dữ liệu chức năng Kiểm thử ứng dụng Mobile (Appium Inspector)

### 1.11.13 Xem báo cáo và lịch sử chạy kiểm thử

|                     |                                                                                                                                                                                                                           |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả</b>        | Người dùng xem toàn bộ lịch sử các lần chạy kiểm thử trong dự án thông qua trang Reports, bao gồm biểu đồ xu hướng, thống kê tỷ lệ Pass/Fail, xem ảnh chụp màn hình (screenshot) khi có lỗi và lọc/tìm kiếm lịch sử chạy. |
| <b>Các tác nhân</b> | Người dùng, Hệ thống                                                                                                                                                                                                      |

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tiền điều kiện</b>    | Đã có ít nhất một lần chạy Test Case hoặc Test Suite được lưu trong .autotest/runs/.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Hậu điều kiện</b>     | Người dùng nắm được tổng quan chất lượng kiểm thử; có thể click vào từng bản ghi để xem chi tiết bước và ảnh chụp màn hình lỗi.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Các luồng sự kiện</b> | <p>Luồng cơ bản:</p> <ol style="list-style-type: none"> <li>1. Người dùng nhấp vào mục "Reports" trong thanh điều hướng hoặc mở tab ReportsView.</li> <li>2. Hệ thống tải toàn bộ bản ghi chạy (ENGINE_GET_ALL_RUNS) và hiển thị: <ul style="list-style-type: none"> <li>- SummaryCards: tổng số run, số Pass, số Fail, tỷ lệ thành công.</li> <li>- StatusDonut: biểu đồ tròn Pass/Fail/Skip.</li> <li>- TrendChart: biểu đồ xu hướng Pass Rate theo thời gian.</li> </ul> </li> <li>3. Người dùng click vào một bản ghi cụ thể để xem RunHistoryPanel: danh sách bước, thời gian từng bước, trạng thái.</li> <li>4. Nếu có screenshot (screenshotMode = final/failure), hệ thống hiển thị ảnh inline trong ScreenshotViewer.</li> <li>5. Người dùng có thể lọc theo: ngày, nền tảng, trạng thái (All/Pass/Fail), tên Test Case.</li> </ol> |



|  |                                                                                                                                                                                                                                                                                                                                                                                       |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>Luồng ngoại lệ:</p> <ol style="list-style-type: none"> <li>1. Chưa có bản ghi nào: Hệ thống hiển thị thông báo "Chưa có dữ liệu. Hãy chạy Test Case đầu tiên."</li> <li>2. Tập bản ghi bị hỏng: Hệ thống bỏ qua bản ghi lỗi và hiển thị cảnh báo.</li> <li>3. Ảnh chụp màn hình không còn tồn tại trên ổ đĩa: Hệ thống hiển thị placeholder "Screenshot not available".</li> </ol> |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Bảng 1.25 – Bảng đặc tả use-case Xem báo cáo và lịch sử chạy kiểm thử

| STT | Trường dữ liệu    | Kiểu dữ liệu | Bắt buộc | Điều kiện hợp lệ                                | Ví dụ                    |
|-----|-------------------|--------------|----------|-------------------------------------------------|--------------------------|
| 1   | Bộ lọc trạng thái | Dropdown     | Không    | All   Passed   Failed   Stopped                 | All                      |
| 2   | Bộ lọc ngày       | Date Range   | Không    | Ngày bắt đầu ≤ ngày kết thúc                    | 2026-06-01 to 2026-06-25 |
| 3   | Tìm kiếm tên Test | Text         | Không    | Tìm kiếm gần đúng theo tên Test Case hoặc Suite | login                    |

| STT | Trường dữ liệu  | Kiểu dữ liệu        | Bắt buộc | Điều kiện hợp lệ                                       | Ví dụ   |
|-----|-----------------|---------------------|----------|--------------------------------------------------------|---------|
| 4   | Screenshot Mode | Dropdown (Settings) | Không    | none   failure (chụp khi lỗi)   final (chụp cuối bước) | failure |

Bảng 1.26 – Dữ liệu chức năng Xem báo cáo và lịch sử chạy kiểm thử

## 1.12 Các yêu cầu phi chức năng

Các yêu cầu phi chức năng định nghĩa các tiêu chuẩn về chất lượng, hiệu năng, bảo mật và giới hạn môi trường mà hệ thống Phần mềm phải tuân thủ để đảm bảo trải nghiệm sử dụng ổn định và đáng tin cậy.

## 1.13 Tính bảo mật

Tuy là một công cụ chạy cục bộ (Local Desktop App), Phần mềm vẫn phải đảm bảo an toàn dữ liệu, đặc biệt là khi tương tác với mã nguồn và môi trường kiểm thử:

Che giấu dữ liệu nhạy cảm: Các thông tin nhạy cảm trong Hồ sơ môi trường (Environment Profiles) như Mật khẩu, API Keys, Access Tokens phải được tự động che giấu (hiển thị dưới dạng dấu \*\*\*) trên giao diện người dùng.

Mã hóa cục bộ: Khuyến nghị các tệp chứa biến môi trường nhạy cảm được mã hóa trước khi lưu xuống ổ cứng để tránh việc rò rỉ dữ liệu nếu tệp bị sao chép trái phép.

Cô lập môi trường chạy (Sandboxing): Các kịch bản kiểm thử (đặc biệt là Web) phải được thực thi trong các phiên trình duyệt ẩn danh hoặc bối cảnh cô lập (Isolated Context của Playwright) để không lưu lại cache, cookie hay ảnh hưởng đến phiên đăng nhập cá nhân của người dùng trên máy tính đó.

An toàn khi sử dụng AI: Tác nhân AI (AI Agent) chỉ được phép đọc mã nguồn trong giới hạn thư mục dự án (Workspace) đã được cấp quyền, ngăn chặn tuyệt đối việc AI tự động truy cập hoặc sửa đổi các tệp tin hệ thống (System Files) của máy chủ chứa.

### 3.14 Tính ràng buộc

Đây là các giới hạn về mặt kỹ thuật, môi trường và kiến trúc mà hệ thống bắt buộc phải phụ thuộc vào:

Ràng buộc về nền tảng Hệ điều hành (OS): Do được đóng gói bằng Electron, phần mềm phải đảm bảo có khả năng chạy độc lập trên 3 hệ điều hành phổ biến: Windows (10/11), macOS (Intel & Apple Silicon) và Linux (Ubuntu/Debian).

Ràng buộc về môi trường thực thi (Dependencies): Để chạy được kiểm thử Web/Desktop, máy người dùng cần có sẵn môi trường Node.js.

- Để chạy được kiểm thử Mobile, máy người dùng buộc phải cài đặt thêm Appium Server, Android SDK (cho Android) hoặc Xcode (cho iOS).

Ràng buộc về cấu hình phần cứng: Do hệ thống chạy đồng thời giao diện React, lõi xử lý, công cụ Test (trình duyệt Chromium) và AI Agent, máy tính yêu cầu cấu hình tối thiểu: CPU 4 cores, RAM 8GB (khuyến nghị RAM 16GB) và ổ cứng SSD để đảm bảo tốc độ đọc/ghi log không bị nghẽn.

Ràng buộc về định dạng dữ liệu (Data Format): Toàn bộ Kịch bản kiểm thử, Bộ định vị (Locator) và Cấu hình dự án bắt buộc phải lưu dưới định dạng chuẩn JSON hoặc YAML. Việc này đảm bảo tính "Single Source of Truth", cho phép người dùng dễ dàng quản lý phiên bản (Version Control) bằng Git mà không gặp lỗi xung đột nhị phân.

### 3.15 Hiệu năng (Performance)

Hệ thống JKAuto phải đáp ứng các chỉ tiêu hiệu năng tối thiểu sau đây nhằm đảm bảo trải nghiệm người dùng mượt mà và không gây gián đoạn quy trình kiểm thử. Các ngưỡng được xác định dựa trên cấu hình phần cứng tối thiểu khuyến nghị (CPU 4 cores, RAM 8 GB, SSD).

| Tiêu chí hiệu năng                        | Ngưỡng mục tiêu | Điều kiện đo lường                                          | Mức độ ưu tiên |
|-------------------------------------------|-----------------|-------------------------------------------------------------|----------------|
| Thời gian khởi động ứng dụng              | < 5 giây        | Máy: CPU 4 cores, RAM 8 GB, SSD; không có dự án mở sẵn      | Cao            |
| Độ trễ phản hồi UI (click, nhập liệu)     | < 100 ms        | Thao tác thông thường trên màn hình soạn thảo Test Case     | Cao            |
| Độ trễ stream log khi chạy test           | < 500 ms        | Test Case 20 bước, kết nối localhost                        | Cao            |
| Thời gian đọc/ghi tệp Test Case           | < 1 giây        | Tệp .test.yaml kích thước $\leq 5$ MB                       | Trung bình     |
| Thời gian mở/load dự án                   | < 3 giây        | Dự án có $\leq 500$ tệp Test Case, Object Repo, Keywords    | Trung bình     |
| Độ trễ phản hồi AI Agent (token đầu tiên) | < 3 giây        | Mô hình local hoặc OpenAI GPT-4o, prompt $\leq 2000$ tokens | Thấp           |

| Tiêu chí hiệu năng                           | Ngưỡng mục tiêu                  | Điều kiện đo lường                                        | Mức độ ưu tiên |
|----------------------------------------------|----------------------------------|-----------------------------------------------------------|----------------|
| Bộ nhớ (RAM) sử dụng khi chạy 1 Test Case    | < 500 MB                         | Test Case web 20 bước, Chrome headless, RAM hệ thống 8 GB | Trung bình     |
| Không rò rỉ bộ nhớ sau 10 lần chạy liên tiếp | Tăng < 50 MB so với lần chạy đầu | Cùng Test Case, chạy liên tiếp 10 lần không restart       | Cao            |

Bảng 1.27 – Chỉ tiêu hiệu năng của hệ thống JKAuto

Trong trường hợp hệ thống chạy đồng thời nhiều Test Suite hoặc sử dụng AI Agent với mô hình lớn, người dùng nên cấu hình máy với RAM  $\geq 16$  GB và CPU  $\geq 8$  cores để đảm bảo hiệu năng ổn định. Các tác vụ nặng như Code Analysis và AI Generation được thực hiện trong luồng nền (background thread) để không chặn giao diện người dùng.

### 3.16 Khả năng tương thích (Compatibility)

Do được đóng gói bằng Electron (Node.js runtime), JKAuto có khả năng chạy đa nền tảng. Bảng dưới đây mô tả chi tiết các hệ điều hành và phiên bản được hỗ trợ chính thức, cùng các ràng buộc đặc thù theo từng môi trường.

| Hệ điều hành | Phiên bản tối thiểu          | Kiến trúc CPU            | Ghi chú / Ràng buộc đặc thù                                                                                    |
|--------------|------------------------------|--------------------------|----------------------------------------------------------------------------------------------------------------|
| Windows      | Windows 10 (build 19041+)    | x86-64                   | Playwright browsers cần cài thủ công nếu đặt ở thư mục ngoài mặc định. Kiểm thử iOS không hỗ trợ trên Windows. |
| Windows      | Windows 11                   | x86-64                   | Phiên bản khuyến nghị cho Windows. Hỗ trợ đầy đủ Web + Desktop + API + Android.                                |
| macOS        | macOS 12 Monterey trở lên    | Intel x86-64             | Hỗ trợ đầy đủ tất cả nền tảng bao gồm iOS Simulator qua Xcode. Yêu cầu Xcode 14+ cho kiểm thử iOS.             |
| macOS        | macOS 12 Monterey trở lên    | Apple Silicon (M1/M2/M3) | Electron và Node.js chạy native arm64. Playwright browsers cần bản arm64. Rosetta 2 không khuyến nghị.         |
| Linux        | Ubuntu 20.04 LTS             | x86-64                   | Yêu cầu cài thêm libgtk, libnss cho Electron. Kiểm thử iOS không hỗ trợ. Android qua ADB được hỗ trợ.          |
| Linux        | Debian 11 (Bullseye) trở lên | x86-64                   | Tương tự Ubuntu. Màn hình desktop (X11 hoặc Wayland) bắt buộc để Electron render UI.                           |

Bảng 1.28 – Ma trận tương thích hệ điều hành của JKAuto

Ngoài hệ điều hành, hệ thống còn phụ thuộc vào một số thành phần môi trường: Node.js 22+ (bắt buộc, kèm theo gói Electron), Playwright browsers (Chromium/Firefox/WebKit – cài qua `npx playwright install`), Appium CLI v2+ và driver tương ứng (UIAutomator2 cho Android, XCUITest cho iOS). Các thành phần này cần được cài đặt đúng phiên bản theo hướng dẫn trong tài liệu cài đặt kèm theo sản phẩm.

## PHẦN II: PHÂN TÍCH HỆ THỐNG

### 2.1 Tổng quan sản phẩm

Phần mềm là IDE kiểm thử tự động theo hướng keyword-driven và file-first. Người dùng không bắt buộc viết trực tiếp mã Playwright hoặc Appium mà có thể tạo từng bước kiểm thử trên giao diện bảng.

Các nhóm người dùng mục tiêu:

- Nhân viên QA cần tạo test case trực quan.
- Lập trình viên cần artifact có thể review bằng Git.
- Nhóm kiểm thử API cần gửi request, assertion và chain token.
- Nhóm Mobile cần Appium hoặc Maestro.
- Người dùng muốn AI hỗ trợ tạo và chỉnh sửa kịch bản.

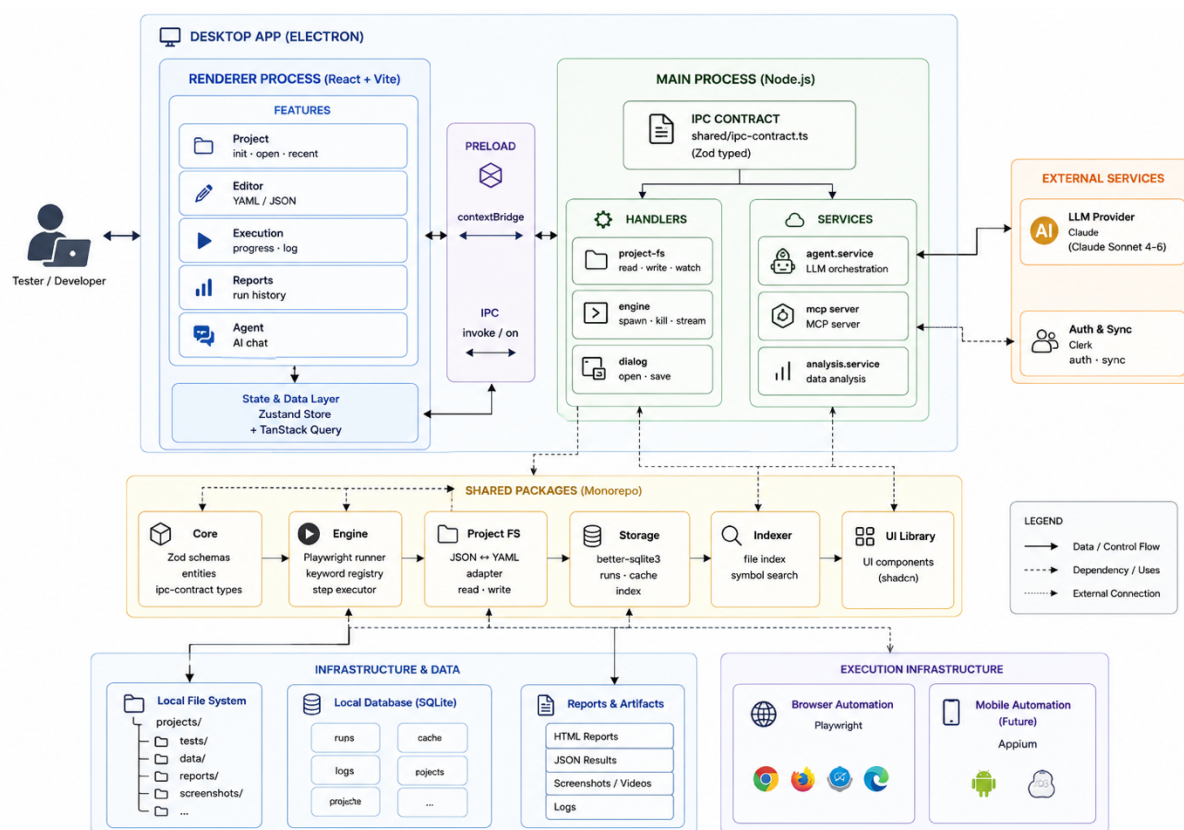
Giá trị cốt lõi của hệ thống:

- Một IDE cho nhiều nền tảng.
- Artifact là tệp văn bản JSON/YAML.
- Keyword metadata là nguồn dữ liệu chung cho editor và engine.
- Tách dữ liệu nguồn khỏi cache và lịch sử chạy.
- Hỗ trợ AI nhưng vẫn có bước kiểm tra schema hoặc review trước khi áp dụng.



## 2.2 Kiến trúc tổng thể

### 2.2.1 Kiến trúc hệ thống



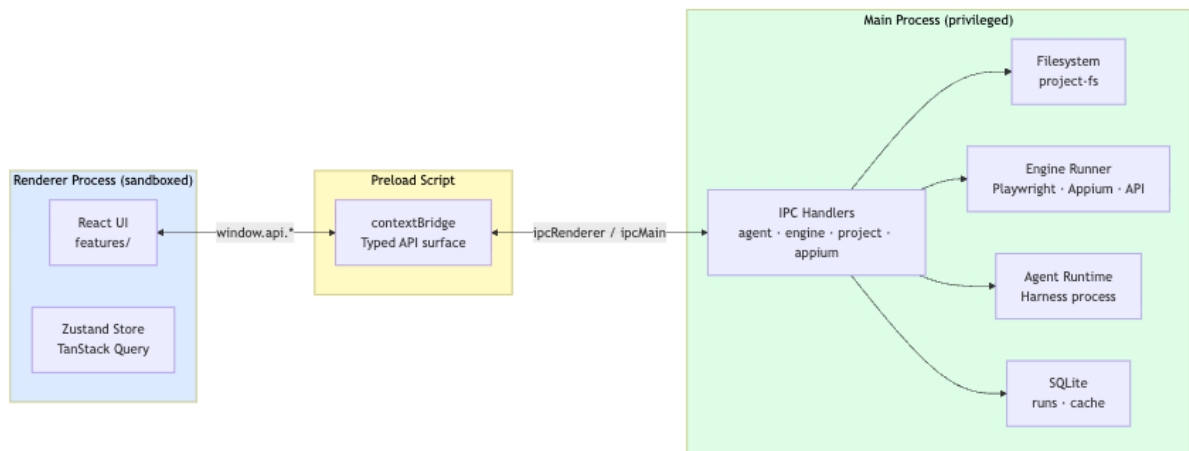
Hình 2.1 — Sơ đồ phụ thuộc giữa các package trong monorepo JKAuto

### 2.2.2 Kiến trúc Electron

Ứng dụng áp dụng mô hình ba lớp chính:

- **Renderer:** giao diện React, quản lý trạng thái hiển thị.
- **Preload:** cầu nối IPC được kiểm soát.
- **Main process:** truy cập filesystem, spawn tiến trình, gọi Appium, HTTP, SQLite và engine.

Nguyên tắc quan trọng là renderer không truy cập trực tiếp filesystem. Các thao tác đặc quyền phải đi qua IPC. Đây là lựa chọn tốt cho cả bảo mật và khả năng kiểm thử.



Hình 2.2 — Kiến trúc ba tầng Electron và ranh giới IPC

### 2.2.3 Kiến trúc Feature theo Vertical Slice

Lớp Renderer của hệ thống được tổ chức theo mô hình **Vertical Slice Architecture**, trong đó mỗi tính năng (feature) được xây dựng như một mô-đun độc lập, chịu trách nhiệm quản lý toàn bộ vòng đời nghiệp vụ của chính nó, bao gồm giao diện người dùng (UI), trạng thái (State), xử lý logic, gọi dịch vụ và các thành phần liên quan. Cách tiếp cận này giúp giảm sự phụ thuộc giữa các tính năng, tăng khả năng mở rộng, bảo trì và cho phép các nhóm phát triển làm việc song song hiệu quả hơn.

Mỗi feature được đặt trong một thư mục riêng biệt bên trong thư mục `renderer/src/features`, chứa đầy đủ các thành phần cần thiết thay vì phân tách theo loại tệp (components, services, hooks, pages,...). Điều này giúp mã nguồn phản ánh trực tiếp các nghiệp vụ của hệ thống.

Các feature chính của hệ thống bao gồm:

- **Project:** Quản lý việc khởi tạo dự án mới, mở dự án hiện có và truy cập danh sách dự án gần đây.
- **Explorer:** Cung cấp cây thư mục và tệp được tối ưu hóa bằng cơ chế Virtualized Tree, hỗ trợ các thao tác quản lý tệp, thư mục và menu ngữ cảnh.
- **Test Cases:** Cho phép xây dựng và chỉnh sửa kịch bản kiểm thử thông qua Step Editor, lựa chọn Keyword, thực thi kiểm thử và theo dõi lịch sử chạy.
- **API Request:** Hỗ trợ tạo và thực thi các yêu cầu HTTP, cấu hình Assertion, lưu lịch sử thực thi và quản lý biến môi trường.

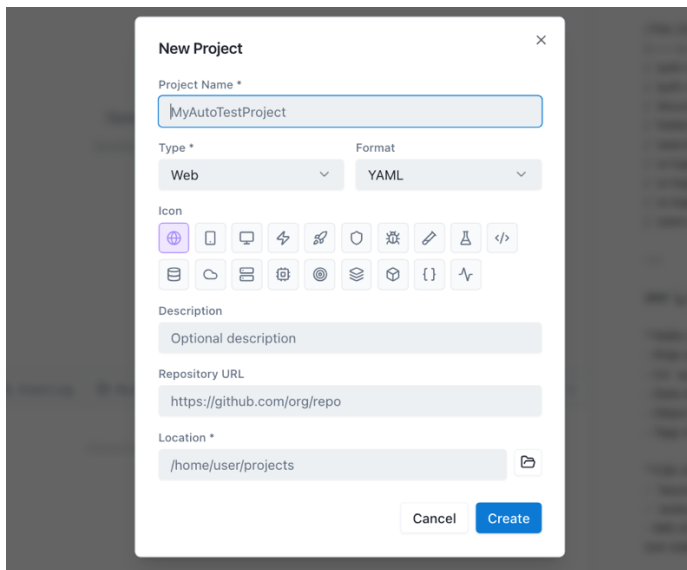
- **Test Suites:** Quản lý tập hợp nhiều Test Case thành các bộ kiểm thử hoàn chỉnh, hỗ trợ chạy hàng loạt và lựa chọn hồ sơ thực thi (Execution Profile).
- **Object Repository:** Quản lý các đối tượng giao diện dùng chung, bao gồm Locator, Selector và các thuộc tính nhận diện phần tử.
- **Keywords:** Cung cấp khả năng tạo và quản lý các Custom Keyword nhằm tái sử dụng logic kiểm thử.
- **Execution:** Hiện thị tiến trình thực thi kiểm thử theo thời gian thực, bao gồm trạng thái chạy và nhật ký hệ thống.
- **Reports:** Quản lý lịch sử thực thi, kết quả từng bước kiểm thử, ảnh chụp màn hình và báo cáo tổng hợp.
- **Appium:** Hỗ trợ quản lý máy chủ Appium, danh sách thiết bị, công cụ Inspector và chức năng phản chiếu màn hình thiết bị di động.
- **Agent:** Cung cấp giao diện tương tác với AI Agent, quản lý phiên làm việc và hiển thị quá trình suy luận cũng như các công cụ được AI sử dụng. Tạo Testcase từ context dự án thông qua thuật toán đồ thị chức năng codegraph và AST.

#### 2.2.3.1 Feature Project

Feature **Project** chịu trách nhiệm quản lý vòng đời của một dự án kiểm thử, bao gồm tạo mới, mở dự án hiện có và quản lý danh sách dự án gần đây.

Các chức năng chính:

- Tạo dự án kiểm thử mới.
- Lựa chọn loại dự án (Web, Mobile, Desktop, API).
- Mở dự án từ hệ thống tập tin.
- Lưu và khôi phục thông tin workspace.
- Quản lý danh sách dự án đã truy cập gần đây.
- Kiểm tra tính hợp lệ của cấu trúc dự án trước khi tải.



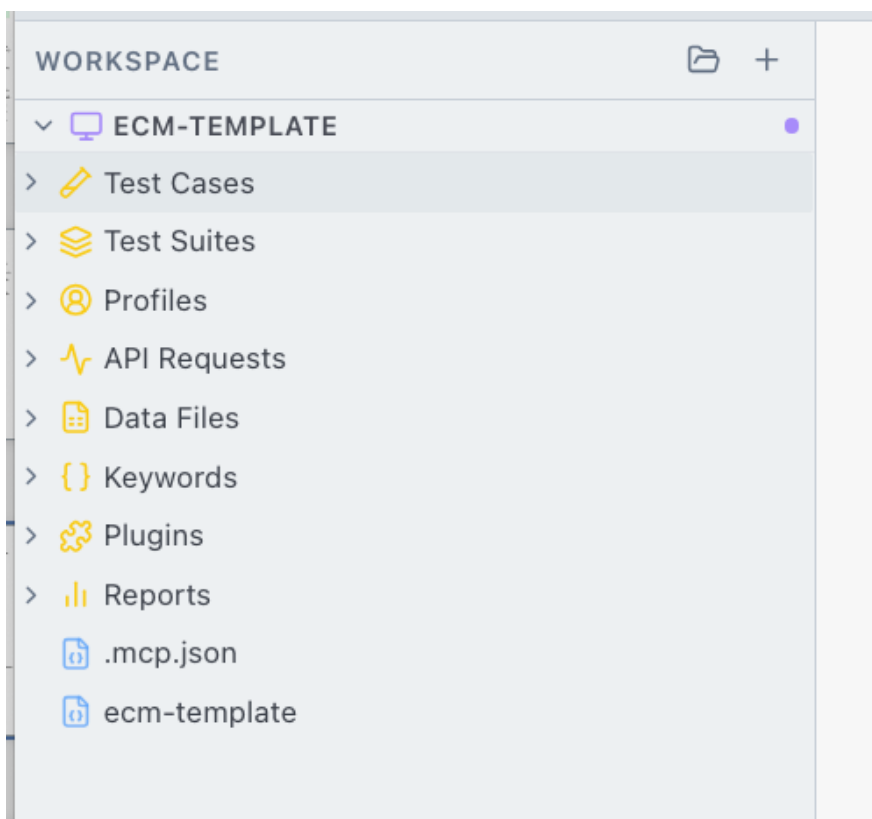
Hình 2.3 : Minh họa chức năng Project

### 2.2.3.2 Feature Explorer

Feature **Explorer** cung cấp khả năng quản lý tài nguyên dự án thông qua giao diện cây thư mục.

Các chức năng chính:

- Hiển thị cấu trúc thư mục và tệp của dự án.
- Hỗ trợ Virtualized Tree nhằm tối ưu hiệu năng với dự án lớn.
- Tạo, đổi tên, sao chép và xóa tệp/thư mục.
- Kéo thả (Drag & Drop) để tổ chức lại cấu trúc dự án.
- Context Menu cho các thao tác nhanh.
- Theo dõi thay đổi của hệ thống tập tin theo thời gian thực.



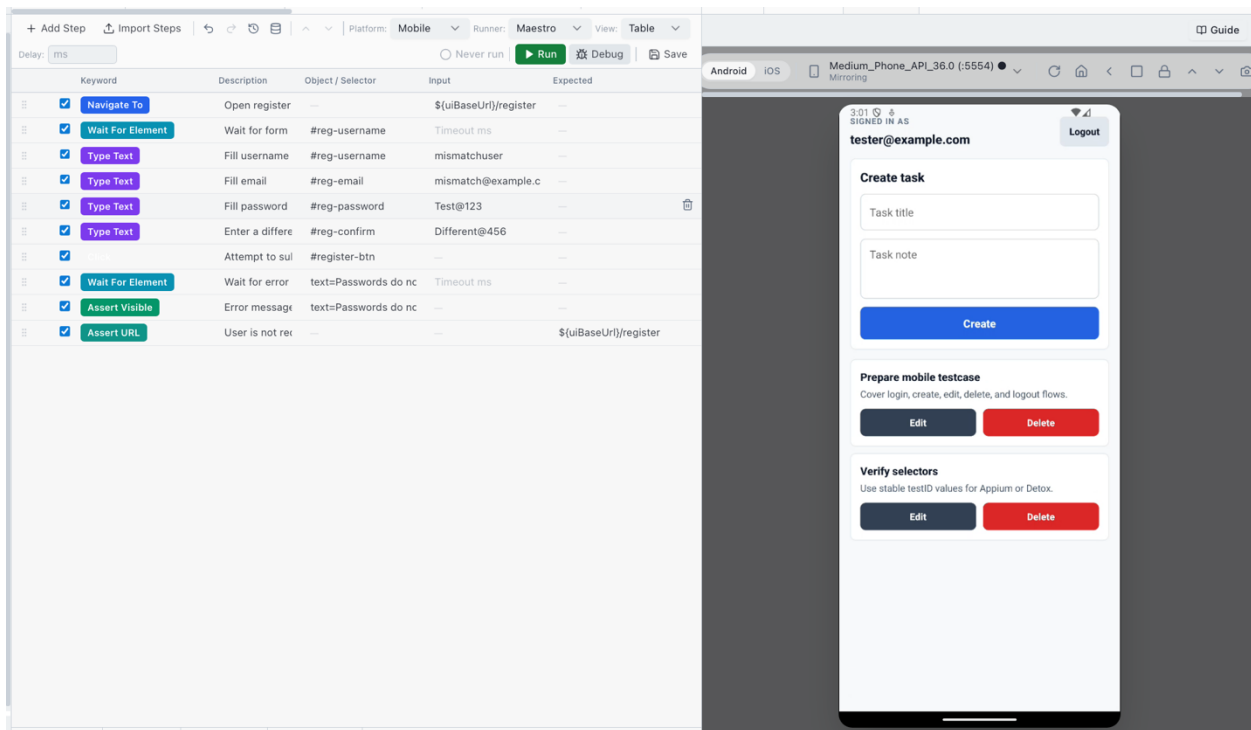
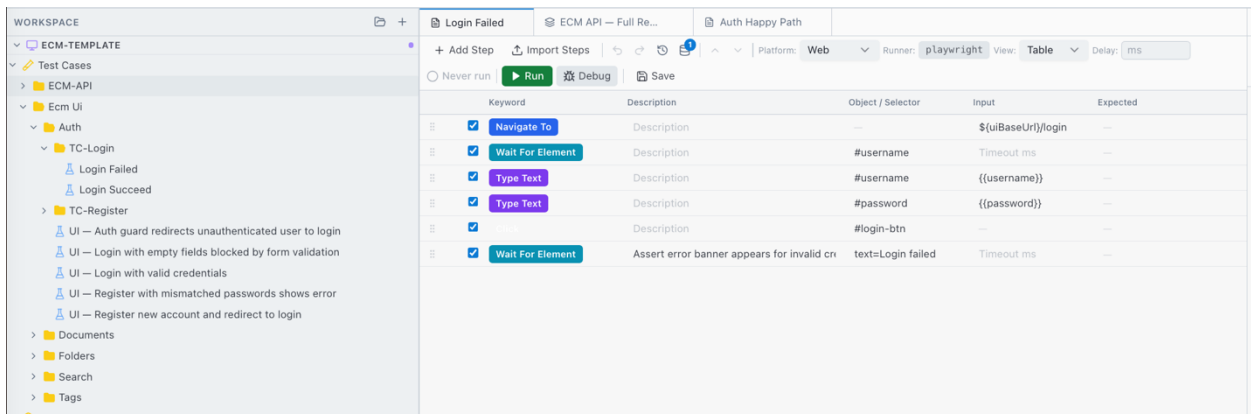
Hình 2.4 : Minh họa chức năng Explorer

### 2.2.3.3 Feature Test Cases

Feature **Test Cases** là thành phần trung tâm của hệ thống, cho phép người dùng xây dựng và quản lý các kịch bản kiểm thử.

Các chức năng chính:

- Tạo và chỉnh sửa Test Case.
- Quản lý danh sách Step kiểm thử.
- Chèn, sao chép và xóa Step.
- Lựa chọn Keyword từ thư viện Keyword.
- Cấu hình dữ liệu đầu vào và dữ liệu xác thực, chạy nhiều lần với bộ dữ liệu.
- Thực thi Test Case độc lập.
- Chạy từ Step được chọn.
- Dừng hoặc tạm dừng quá trình thực thi.
- Xem lịch sử chỉnh sửa và thực thi.



Hình 2.5 : Minh họa chức năng Test Cases

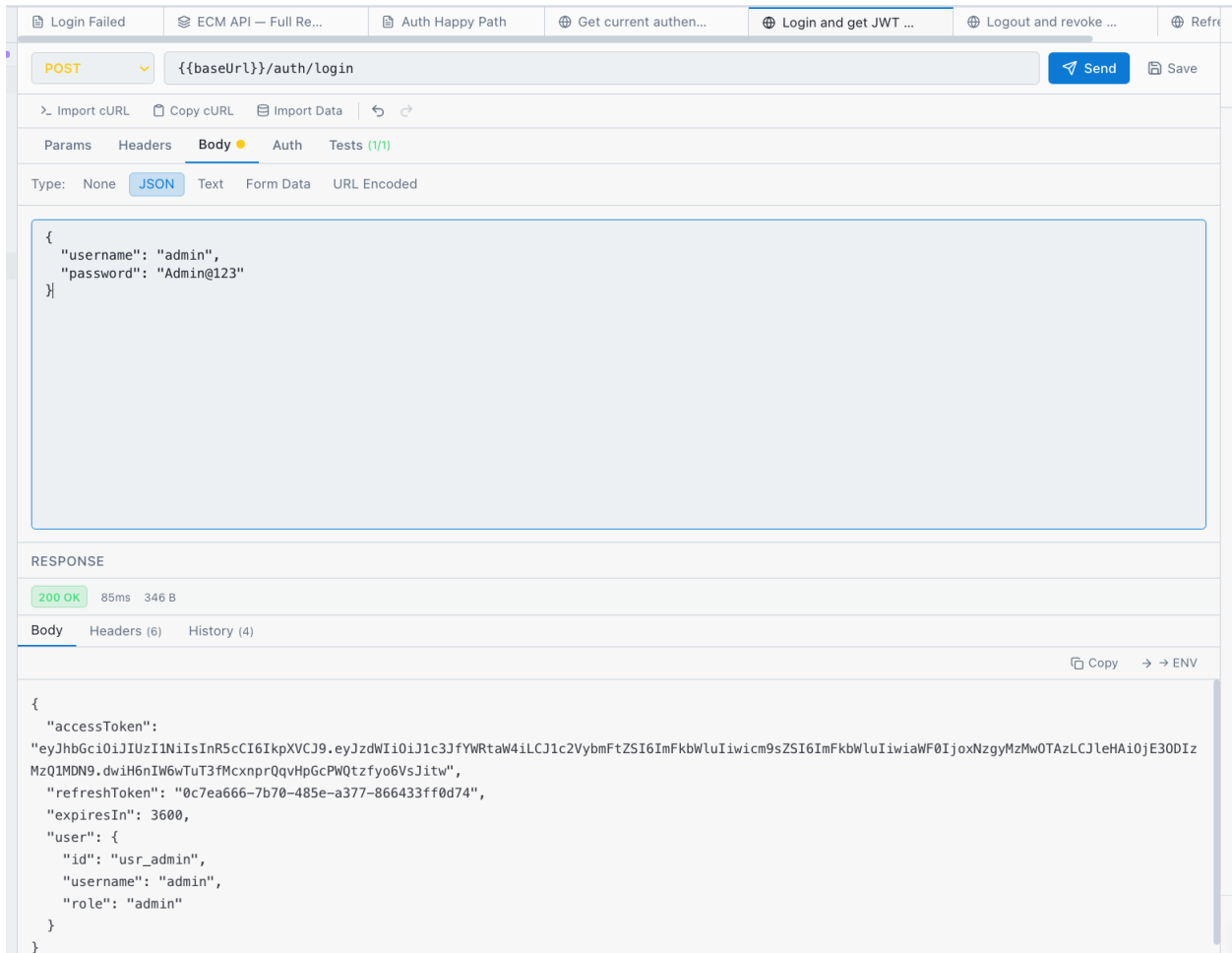
### 2.2.3.4 Feature API Request

Feature **API Request** hỗ trợ kiểm thử dịch vụ API trực tiếp bên trong ứng dụng.

Các chức năng chính:

- Tạo yêu cầu HTTP (GET, POST, PUT, PATCH, DELETE).
- Cấu hình Header, Query Parameters và Request Body.
- Hỗ trợ JSON, Form Data và Multipart Data.
- Xem phản hồi API theo thời gian thực.

- Tạo Assertion cho mã trạng thái và dữ liệu phản hồi.
- Lưu lịch sử thực thi API.
- Lưu giá trị phản hồi vào biến môi trường.
- Tái sử dụng API Request trong các Test Case.



Hình 2.6 : Minh họa chức năng API Request

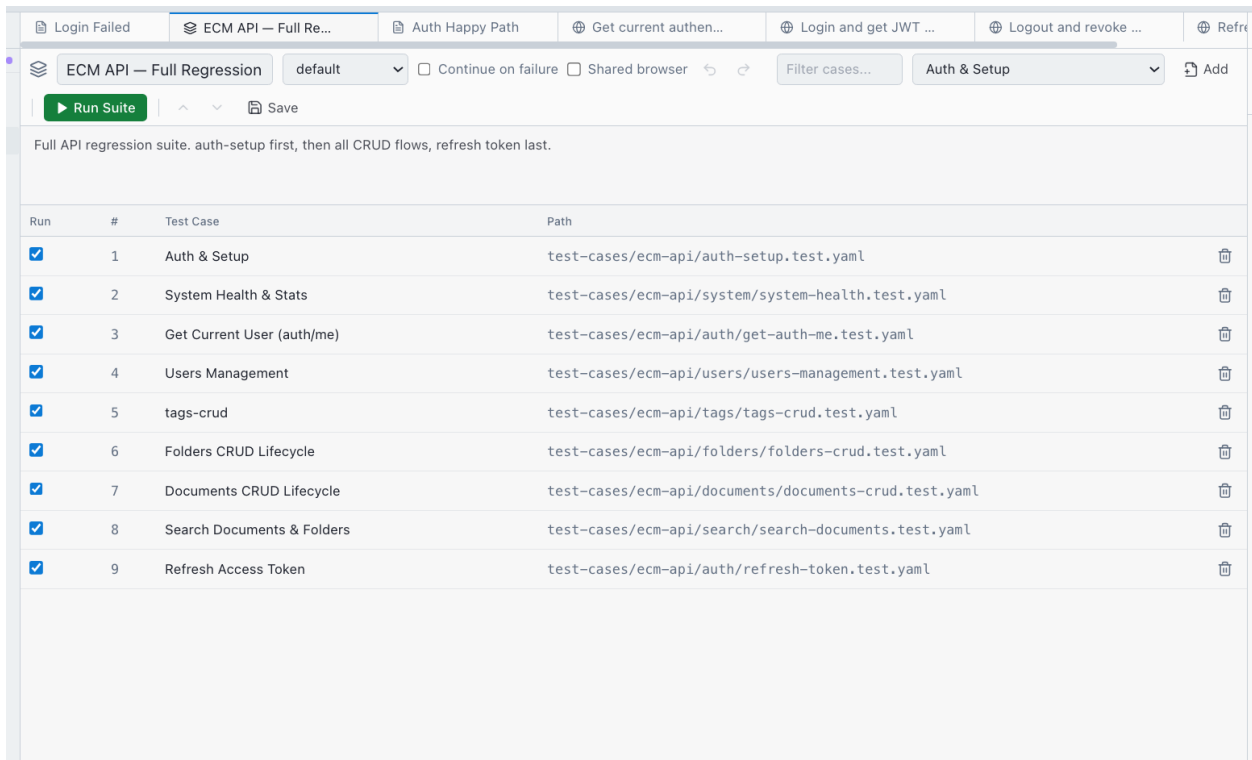
### 2.2.3.5 Feature Test Suites

Feature **Test Suites** cho phép tổ chức nhiều Test Case thành một bộ kiểm thử hoàn chỉnh.

Các chức năng chính:

- Tạo và chỉnh sửa Test Suite.
- Thêm hoặc loại bỏ Test Case khỏi Suite.
- Sắp xếp thứ tự thực thi.

- Chạy toàn bộ Suite.
- Chạy theo nhóm hoặc theo Tag.
- Hỗ trợ Continue on Failure.
- Hỗ trợ Shared Browser Session.
- Quản lý Execution Profile.
- Tổng hợp kết quả thực thi của toàn bộ Suite.



Hình 2.7 : Minh họa chức năng Test Suites

### 2.2.3.6 Feature Keywords

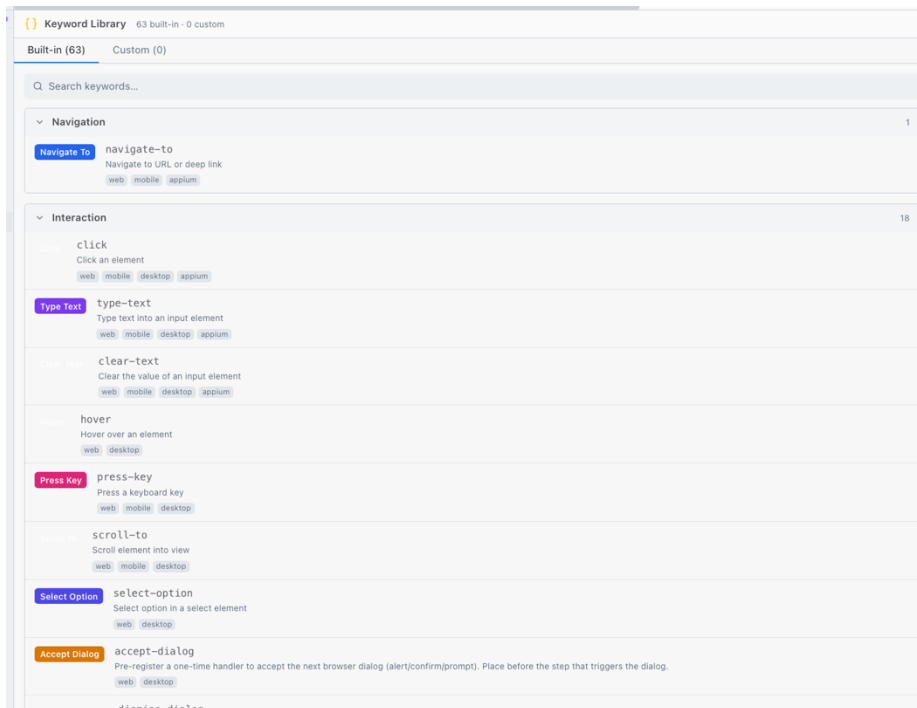
Feature **Keywords** cung cấp khả năng xây dựng các hành động kiểm thử có thể tái sử dụng.

Các chức năng chính:

- Tạo Custom Keyword.
- Định nghĩa Input và Output Parameter.
- Quản lý thư viện Keyword dùng chung.
- Tổ chức Keyword theo nhóm chức năng.
- Gọi Keyword trong Test Case.
- Hỗ trợ tái sử dụng logic kiểm thử.



- Giám trùng lặp mã nguồn và thao tác.



Hình 2.8 : Minh họa chức năng Keywords

### 2.2.3.7 Feature Object Repository

Feature **Object Repository** quản lý các đối tượng giao diện được sử dụng trong kiểm thử tự động.

Các chức năng chính:

- Tạo và quản lý Object Repository.
- Quản lý Locator và Selector.
- Hỗ trợ CSS Selector, XPath và Accessibility ID.
- Kiểm tra Locator trực tiếp trên giao diện.
- Tái sử dụng đối tượng giữa nhiều Test Case.
- Cập nhật hàng loạt khi giao diện thay đổi

### 2.2.3.8 Feature Execution

Feature **Execution** đảm nhiệm việc giám sát quá trình thực thi kiểm thử.

Các chức năng chính:

- Hiển thị trạng thái thực thi theo thời gian thực.
- Theo dõi tiến độ chạy Test Case.
- Hiển thị nhật ký hệ thống (Realtime Log).
- Hiển thị lỗi và Stack Trace.
- Hỗ trợ dừng hoặc hủy phiên kiểm thử.
- Quản lý nhiều tiến trình thực thi đồng thời.
- Cung cấp thống kê Pass, Fail và Skip.

### 2.2.3.9 Feature Reports

Feature **Reports** chịu trách nhiệm tổng hợp và hiển thị kết quả kiểm thử.

Các chức năng chính:

- Lưu trữ lịch sử chạy kiểm thử.
- Hiển thị kết quả Pass/Fail.
- Xem chi tiết từng bước thực thi.
- Hiển thị ảnh chụp màn hình lỗi.
- Xuất báo cáo dưới nhiều định dạng.
- Tổng hợp thống kê theo thời gian.
- So sánh kết quả giữa các lần thực thi.

### 2.2.3.10 Feature Appium

Feature **Appium** hỗ trợ kiểm thử tự động trên thiết bị di động.

Các chức năng chính:

- Quản lý Appium Server.
- Khởi động và dừng Appium Server.
- Quản lý danh sách thiết bị Android và iOS.
- Kết nối thiết bị thật hoặc giả lập.
- Hiển thị màn hình thiết bị theo thời gian thực.
- Tích hợp Appium Inspector.

- Kiểm tra và ghi nhận Locator trên thiết bị.
- Quản lý phiên làm việc với thiết bị.

### 2.2.3.11 Feature Agent

Feature **Agent** tích hợp trí tuệ nhân tạo nhằm hỗ trợ người dùng trong quá trình xây dựng và kiểm thử hệ thống.

Các chức năng chính:

- Giao tiếp với AI Agent thông qua Chat Interface.
- Quản lý phiên làm việc của Agent.
- Hiển thị quá trình suy luận (Reasoning).
- Hiển thị danh sách Tool được Agent sử dụng.
- Hỗ trợ phân tích mã nguồn.
- Sinh Test Case tự động.
- Đề xuất chiến lược kiểm thử.
- Hỗ trợ sửa lỗi và tối ưu kịch bản kiểm thử.

## 2.3 Công nghệ sử dụng

| Thành phần     | Công nghệ                 | Lý do lựa chọn                                       |
|----------------|---------------------------|------------------------------------------------------|
| Desktop shell  | Electron                  | FS access, native dialog, spawn process, đa nền tảng |
| Build renderer | Vite + electron-vite      | HMR nhanh, ESM, bundle tối ưu Electron               |
| Giao diện      | React 18 + TypeScript 5.x | Ecosystem lớn, type safety đầu cuối                  |

| Thành phần           | Công nghệ                         | Lý do lựa chọn                                      |
|----------------------|-----------------------------------|-----------------------------------------------------|
| Thành phần UI        | shadcn/ui, Radix UI, Tailwind CSS | Accessible, composable, không lock vendor           |
| Cây tệp              | react-arborist                    | Virtualized, drag-drop, rename inline               |
| Resize panes         | react-resizable-panels            | shadcn ecosystem, persist layout                    |
| State UI             | Zustand                           | Lightweight, không boilerplate Redux                |
| Data fetching        | TanStack Query v5                 | Cache, invalidation, IPC bridge                     |
| Schema / validation  | Zod v3                            | Runtime validation + TypeScript inference từ schema |
| Data format          | JSON, YAML (yaml package)         | Git-friendly; YAML giữ comment                      |
| Web / Desktop runner | @playwright/test                  | Stable selector, trace, screenshot, multi-browser   |

| Thành phần    | Công nghệ                                     | Lý do lựa chọn                                    |
|---------------|-----------------------------------------------|---------------------------------------------------|
| Native Mobile | Appium / WebDriverIO                          | Industry standard cho native mobile automation    |
| Mobile DSL    | Maestro keyword mapping                       | Bridge JKAuto keyword sang Maestro flow           |
| Database      | better-sqlite3                                | Sync API, không cần ORM, nhanh, cùng runtime Node |
| AI backend    | Agent Runtime Harness (nội bộ)                | Session, tool loop agentic, MCP, SSE streaming    |
| AI SDK        | Vercel AI SDK + MCP protocol                  | Tool calling, streaming, model-agnostic           |
| Repo analysis | TypeScript ESTree, simple-git, Swagger Parser | AST parse đa ngôn ngữ, clone nhanh, API spec      |
| Monorepo      | pnpm workspace + Turborepo                    | Fast install, build cache, task graph parallel    |

## **2.4 Phạm vi kiểm thử**

### **2.4.1 Những chức năng được kiểm thử**

Các module cốt lõi của nền tảng Phần mềm sẽ được đội ngũ QA/Tester tiến hành kiểm thử toàn diện, đảm bảo hệ thống đáp ứng đúng yêu cầu nghiệp vụ:

Module Khởi tạo và Quản lý dự án (Workspace Management):

- Kiểm tra chức năng tạo mới dự án, mở dự án có sẵn.
- Kiểm tra tính toàn vẹn của cấu trúc thư mục dự án trên hệ thống tệp tin.
- Kiểm tra chức năng lưu/tải cấu hình (Settings) và hồ sơ môi trường (Profiles).

Module Soạn thảo Kịch bản (Test Authoring):

- Kiểm tra giao diện soạn thảo (Editor), tính năng kéo thả từ khóa (Keywords).
- Kiểm tra chức năng xác thực (Validation) các trường bắt buộc (Tên, Nền tảng, Timeout).
- Kiểm tra khả năng lưu trữ kịch bản dưới định dạng chuẩn (JSON/YAML).

Module Tương tác API:

- Kiểm tra chức năng gửi yêu cầu (Request) với các phương thức (GET, POST, PUT, DELETE).
- Kiểm tra chức năng import API từ cURL/Swagger.
- Kiểm tra việc xử lý dữ liệu phản hồi (Response) và các bộ kiểm tra (Assertions).

Module Trợ lý AI (Autogen):

- Kiểm tra chức năng quét mã nguồn (Indexing) từ đường dẫn được cung cấp.
- Kiểm tra độ chính xác và định dạng của các Test Case được AI sinh ra.

#### Module Thực thi & Báo cáo (Execution & Reporting):

- Kiểm tra luồng giao tiếp IPC giữa giao diện và Engine thực thi.
- Kiểm tra khả năng kết nối tới các trình duyệt (Playwright) và trình giả lập (Appium).
- Kiểm tra tính chính xác của báo cáo kết quả (Pass/Fail) và việc chụp ảnh màn hình khi có lỗi.

### 2.4.2 Những chức năng chưa được kiểm thử

Để tập trung tối ưu hóa nguồn lực cho các chức năng trọng yếu, một số thành phần sau đây sẽ không nằm trong phạm vi kiểm thử của đợt này:

Tính năng tự động cập nhật phần mềm (Auto-Update): Quy trình kiểm tra bản vá và tải xuống tự động qua server từ xa sẽ được bỏ qua trong giai đoạn này.

Các trình điều khiển (Drivers/Adapters) bên thứ ba: Không kiểm thử các lỗi xuất phát từ bản thân phần mềm Appium, Playwright, hay Selenium. Mọi lỗi phát sinh từ các thư viện này sẽ được coi là nằm ngoài tầm kiểm soát của Phần mềm.

Tính năng đồng bộ hóa đám mây (Cloud Sync): Các chức năng đồng bộ dự án lên dịch vụ Cloud/Database trực tuyến chưa được phát triển và chưa nằm trong phạm vi kiểm thử.

Kiểm thử hiệu năng (Stress/Load Test) trên diện rộng: Không kiểm thử hệ thống trong các trường hợp chạy hàng ngàn kịch bản cùng một lúc với các tác vụ gây tải nặng nề vượt quá cấu hình phần cứng khuyến nghị.

Bảo mật mạng chuyên sâu: Không thực hiện kiểm thử xâm nhập (Penetration Testing) đối với các giao tiếp mạng vì ứng dụng tập trung vào tính chất hoạt động cục bộ (Offline-first).

## 2.5 Chiến lược kiểm thử

Chiến lược kiểm thử (test strategy) là một phần quan trọng của quá trình phát triển phần mềm và đảm bảo chất lượng sản phẩm. Có một chiến lược kiểm thử rõ ràng và hiệu quả giúp các nhóm phát triển phần mềm đạt được nhiều lợi ích, bao gồm: Đảm bảo chất lượng sản phẩm: Chiến lược kiểm thử giúp xác định các loại kiểm thử cần thực hiện, các công cụ cần sử dụng, và quy trình kiểm thử. Điều này giúp phát hiện và sửa chữa lỗi sớm, đảm bảo rằng sản phẩm cuối cùng đáp ứng các yêu cầu về chất lượng và hoạt động như mong đợi.

Tiết kiệm thời gian và chi phí: Giúp giảm thiểu thời gian và chi phí phát hiện và sửa lỗi sau khi sản phẩm đã phát hành. Phát hiện và sửa lỗi trong giai đoạn phát triển sớm thường ít tốn kém hơn nhiều so với việc xử lý chúng sau khi phần mềm đã được triển khai.

Định hướng rõ ràng: Cung cấp hướng dẫn rõ ràng về các phương pháp và quy trình kiểm thử, giúp đội ngũ kiểm thử hiểu rõ vai trò và nhiệm vụ của mình. Điều này giúp giảm thiểu sai sót và tăng hiệu quả làm việc.

Quản lý rủi ro: Giúp xác định các khu vực có nguy cơ cao của phần mềm và tập trung kiểm thử vào những phần này. Điều này giúp giảm thiểu rủi ro khi triển khai sản phẩm và đảm bảo rằng các chức năng quan trọng nhất hoạt động chính xác.

Đánh giá và cải tiến liên tục: Bao gồm các biện pháp đánh giá và cải tiến liên tục. Điều này giúp nhóm phát triển nhận ra các điểm yếu trong quy trình kiểm thử và cải tiến chúng để nâng cao chất lượng sản phẩm qua từng phiên bản..

### 2.5.1 Kiểm thử đa nền tảng

Do Phần mềm được xây dựng trên nền tảng Electron, mục tiêu là đảm bảo khả năng chạy độc lập và ổn định trên ba hệ điều hành phổ biến nhất hiện nay là Windows, macOS và Linux. Sự khác biệt về kiến trúc hệ thống (như quản lý tệp tin, quyền truy cập thư mục, và cách thức giao tiếp với Driver phần cứng) giữa các nền tảng này có thể dẫn đến những sai khác trong quá trình thực thi kịch bản kiểm thử.

Vì vậy, việc kiểm thử đa nền tảng cho Phần mềm là yêu cầu bắt buộc nhằm đảm bảo:



Tính nhất quán: Các thao tác soạn thảo, khởi tạo dự án và cấu hình môi trường phải hoạt động đồng nhất trên cả Windows, macOS và Linux.

Tính tương thích của Driver: Đảm bảo hệ thống có khả năng nhận diện và giao tiếp chính xác với các Driver/Adapter tương ứng trên từng hệ điều hành (ví dụ: nhận diện đúng đường dẫn Android SDK trên Windows hoặc Xcode trên macOS).

Quản lý tài nguyên: Kiểm soát hành vi tiêu thụ tài nguyên của ứng dụng (RAM, CPU) trên mỗi hệ điều hành khác nhau để đảm bảo Phần mềm không gặp lỗi treo hoặc giật lag khi xử lý các tác vụ kiểm thử nặng.

Việc lên kế hoạch kiểm thử trên tất cả các nền tảng mục tiêu này giúp đội ngũ phát triển phát hiện sớm các vấn đề về tương thích, đảm bảo trải nghiệm người dùng không bị gián đoạn bất kể họ đang sử dụng môi trường làm việc nào.

### **2.5.2 Kiểm thử đơn vị**

#### **\* Mục đích**

Kiểm thử đơn vị là cấp độ kiểm thử cơ sở nhất trong dự án Phần mềm, tập trung vào việc xác minh tính đúng đắn của từng hàm, module hoặc class riêng lẻ trước khi chúng được kết hợp thành hệ thống. Mục tiêu chính bao gồm:

Xác minh tính logic: Đảm bảo các hàm xử lý logic cốt lõi (như hàm phân tích cú pháp tệp JSON/YAML, hàm xử lý chuỗi từ khóa, hoặc các hàm tính toán logic của AI Agent) hoạt động chính xác theo yêu cầu thiết kế.

Nâng cao chất lượng mã nguồn: Buộc lập trình viên phải viết mã có cấu trúc chặt chẽ, dễ kiểm thử và dễ bảo trì hơn ngay từ giai đoạn phát triển.

Hỗ trợ bảo trì dài hạn: Đảm bảo khi hệ thống có thay đổi hoặc nâng cấp tính năng mới, các đơn vị mã nguồn cũ vẫn hoạt động đúng, ngăn chặn các lỗi phát sinh ngoài ý muốn (regression) ngay tại cấp độ hàm.

#### **\* Đặc điểm của kiểm thử đơn vị trong dự án Phần mềm:**

Phạm vi: Tập trung vào các đơn vị nhỏ nhất trong cấu trúc phần mềm (hàm xử lý, class quản lý trạng thái, service gọi API).

Đối tượng thực hiện: Do chính các lập trình viên của nhóm thực hiện trong quá trình code, thường được tích hợp ngay trong môi trường phát triển (VS Code).

Tự động hóa: Toàn bộ các bài kiểm thử đơn vị được thực hiện tự động thông qua các công cụ như Jest hoặc Vitest. Điều này giúp việc chạy lại các kịch bản kiểm thử trở nên nhanh chóng mỗi khi mã nguồn được thay đổi.

Tính chất: Sử dụng phương pháp Mocking để giả lập các dữ liệu đầu vào phức tạp hoặc các phản hồi từ AI, giúp tách biệt đơn vị cần kiểm thử khỏi môi trường thực tế, đảm bảo kết quả kiểm thử mang tính độc lập và chính xác cao.

### **2.5.3 Kiểm thử tích hợp**

\* Mục đích

Kiểm thử tích hợp trong dự án Phần mềm nhằm mục đích xác minh sự tương tác và luồng truyền tải dữ liệu giữa các module độc lập sau khi đã vượt qua kiểm thử đơn vị. Mục tiêu trọng yếu bao gồm:

Kiểm tra luồng dữ liệu (Data Flow): Đảm bảo dữ liệu từ giao diện người dùng (React UI) được truyền tải chính xác qua giao thức IPC (Inter-Process Communication) đến các tiến trình lõi (Engine).

Phát hiện lỗi giao tiếp: Xác định các điểm nghẽn hoặc lỗi định dạng dữ liệu phát sinh khi kết nối giữa module quản lý dự án, module soạn thảo và module thực thi.

Đảm bảo tính toàn vẹn hệ thống: Xác nhận rằng khi các module được kết hợp, Phần mềm vận hành như một thực thể thống nhất, không xảy ra xung đột khi gọi các Adapter (Playwright, Appium) từ lõi Engine.

\* Đặc điểm kiểm thử tích hợp trong dự án Phần mềm:

Trọng tâm module: Thay vì kiểm tra logic bên trong từng hàm, kiểm thử tích hợp tập trung vào các Giao diện kết nối (Interfaces). Ví dụ: Kiểm tra xem Main Process có nhận được đúng cấu trúc kịch bản từ Renderer Process hay không.

Phương pháp tiếp cận:

- Tiếp cận từ dưới lên (Bottom-Up): Ưu tiên kiểm tra các module lõi như Test Engine và Drivers trước, sau đó mới tích hợp với giao diện. Điều này đảm bảo nền tảng thực thi ổn định là bước đệm vững chắc cho giao diện người dùng.
- Tiếp cận kết hợp (Sandwich/Hybrid): Sử dụng các Mock/Stub để giả lập giao diện khi các module Engine đang được hoàn thiện, giúp rút ngắn thời gian phát triển và tích hợp.

Tự động hóa: Các kịch bản kiểm thử tích hợp được tự động hóa hoàn toàn bằng Playwright và Jest. Hệ thống sẽ tự động thực thi lại các bài kiểm tra này mỗi khi có mã nguồn mới được đẩy lên (commit) hoặc khi các module mới (ví dụ: thêm một Adapter cho trình duyệt mới) được tích hợp vào nền tảng.

Phát hiện lỗi sớm: Việc thực hiện tích hợp liên tục giúp đội ngũ dự án phát hiện ngay các lỗi về sai lệch cấu trúc dữ liệu (JSON/YAML) hoặc xung đột thư viện ngay từ giai đoạn sớm, tránh việc phải sửa lỗi ở giai đoạn cuối dự án.

## 2.5.4 Kiểm thử hệ thống

\* Mục đích

Kiểm thử hệ thống là giai đoạn kiểm chứng toàn diện nền tảng Phần mềm như một thực thể hoàn chỉnh, đảm bảo hệ thống đáp ứng đầy đủ các yêu cầu chức năng và phi chức năng đã được thiết lập. Mục tiêu bao gồm:

Xác minh quy trình End-to-End: Kiểm tra toàn bộ vòng đời của một dự án kiểm thử, từ khâu khởi tạo không gian làm việc (Workspace), soạn thảo kịch bản, cho đến khâu thực thi kiểm thử và xuất báo cáo kết quả.

Đánh giá toàn diện: Kiểm tra tính chính xác của các chức năng, đồng thời đo lường các thuộc tính phi chức năng như hiệu năng xử lý (latency của Engine), độ an toàn dữ liệu và khả năng tương thích trên các hệ điều hành mục tiêu.

\* Đặc điểm kiểm thử hệ thống trong dự án Phần mềm:

Phạm vi toàn diện: Bao phủ tất cả các module của Phần mềm, đảm bảo không có tính năng nào bị bỏ sót trong quá trình kiểm tra tích hợp.

Kiểm thử dựa trên yêu cầu (Requirement-based): Mọi kịch bản kiểm thử hệ thống đều được ánh xạ trực tiếp từ tài liệu Đặc tả yêu cầu phần mềm (SRS). Đảm bảo mỗi tính năng người dùng mong đợi đều được kiểm chứng thông qua các kịch bản thực tế.

Môi trường mô phỏng thực tế: Được thực hiện trong môi trường mô phỏng các điều kiện sử dụng thực tế của kỹ sư QA: cấu hình máy tính đa dạng, kết nối Internet không ổn định, và việc chạy các kịch bản kiểm thử trên các phần mềm mục tiêu (Web bán hàng, App mobile mẫu).

Kết hợp chức năng và phi chức năng:

- Chức năng: Xác minh kết quả "Passed/Failed" của Test Case có khớp với thực tế chạy trên phần mềm mục tiêu hay không.
- Phi chức năng: Kiểm tra khả năng chịu tải của trình thực thi khi chạy đồng thời nhiều tác vụ (multithreading/parallel execution), và sự ổn định của giao diện khi hiển thị hàng ngàn dòng log.

Đội ngũ kiểm thử độc lập & Phương pháp kết hợp: Được thực hiện bởi đội ngũ QA độc lập (người không tham gia vào quá trình code module đó để đảm bảo tính khách quan). Chiến lược kết hợp giữa Kiểm thử tự động (sử dụng chính Phần mềm để tự test chính nó – Self-testing) và Kiểm thử thủ công (kiểm tra trải nghiệm UX trên giao diện người dùng) để đảm bảo chất lượng cao nhất.

### **2.5.5 Kiểm thử hồi quy**

\* Mục đích

Kiểm thử hồi quy đóng vai trò là "chốt chặn" an toàn cho quá trình phát triển nền tảng Phần mềm. Mục tiêu chính bao gồm:

Bảo toàn tính toàn vẹn: Đảm bảo rằng sau khi thêm các tính năng mới (như thêm Adapter cho trình duyệt mới, hoặc nâng cấp module AI), các chức năng cũ (như soạn thảo kịch bản, cấu hình API) vẫn vận hành chính xác.

Xác nhận tính ổn định: Xác nhận hệ thống không bị lỗi (bugs) cũ quay lại sau khi thực hiện các thay đổi mã nguồn hoặc cập nhật thư viện phụ thuộc (dependencies).

Phát hiện lỗi mới phát sinh: Ngăn chặn việc các đoạn mã mới can thiệp hoặc làm hỏng logic của các module đang hoạt động tốt.

\* Đặc điểm kiểm thử hồi quy trong dự án Phần mềm:

Do bản chất Phần mềm là một nền tảng thiết kế và thực thi kiểm thử tự động đa nền tảng, nhóm phát triển đã áp dụng chiến lược sử dụng chính Phần mềm để kiểm thử lại hệ thống Phần mềm sau mỗi lần nâng cấp (Release) hoặc cập nhật mã nguồn. Thay vì phụ thuộc hoàn toàn vào các framework bên ngoài, hệ thống sẽ tự đọc và chạy các kịch bản (Test Cases) được viết bằng định dạng JSON/YAML của chính nó.

Chiến lược Self-testing được triển khai thông qua các kịch bản cụ thể sau:

- Tự kiểm thử module API (API Self-Testing): Nhóm tạo ra các kịch bản Test Case API trực tiếp trên Phần mềm. Trình thực thi (Engine) của Phần mềm sẽ đọc các kịch bản này, tự động giả lập các HTTP Request (GET, POST, PUT, DELETE) gửi đến các máy chủ thử nghiệm (hoặc chính backend cục bộ nếu có). Sau đó, module Assertions của Phần mềm sẽ tự động đánh giá Header, Body và Status Code trả về. Quá trình này giúp xác minh module "Tương tác API" đang hoạt động hoàn hảo mà không cần mở Postman hay Insomnia.
- Tự kiểm thử luồng Hệ thống tệp (File System & Workspace): Kịch bản kiểm thử được thiết lập để yêu cầu Phần mềm tự động tạo một dự án mới (Project), ghi dữ liệu kịch bản xuống tệp .json/.yaml, sau đó đọc ngược lại tệp đó để kiểm tra tính toàn vẹn của dữ liệu. Việc này đảm bảo luồng đọc/ghi cấu hình (Profile/Settings) của hệ thống không bị lỗi hỏng (corrupt) sau các bản cập nhật.

- Tự kiểm thử giao diện Desktop (UI Self-Testing): Tận dụng bộ chuyển đổi (Adapter) Playwright được tích hợp sẵn, Phần mềm có khả năng khởi chạy một phiên bản (instance) thứ hai của chính nó. Phiên bản "kiểm thử" sẽ ra lệnh click vào các nút bấm như "Tạo Test Case", "Chạy Test" trên phiên bản "mục tiêu" để xác minh giao diện React/Electron phản hồi chính xác, không bị vỡ bố cục hay lỗi chức năng.

### **Lợi ích của chiến lược Self-testing:**

- Tiết kiệm tối đa tài nguyên: Biến các kịch bản tạo ra trong quá trình phát triển thành một bộ Kiểm thử hồi quy tự động (Automated Regression Suite). Đội ngũ QA không cần phải tốn công viết lại kịch bản trên các công cụ khác.
- Minh chứng thực tế cho chất lượng sản phẩm (Proof of Capability): Việc Phần mềm có thể tự tìm ra lỗi của chính mình là minh chứng thuyết phục nhất cho năng lực, độ ổn định và tính thực tiễn của nền tảng trước hội đồng đánh giá.

### **2.5.6 Kiểm thử hiệu năng**

#### **\* Mục đích**

Đối với nền tảng Phần mềm, kiểm thử hiệu năng không chỉ là đo tốc độ, mà là đảm bảo hệ thống có khả năng xử lý các kịch bản kiểm thử phức tạp mà không làm ảnh hưởng đến hiệu suất của máy tính người dùng. Mục tiêu trọng yếu bao gồm:

Tối ưu hóa tài nguyên: Đảm bảo ứng dụng sử dụng RAM/CPU hợp lý, tránh tình trạng ứng dụng "chiếm dụng" tài nguyên gây treo máy.

Xác định ngưỡng chịu tải: Tìm ra giới hạn của số lượng Steps hoặc Test Cases mà một dự án có thể chứa trước khi trình soạn thảo hoặc trình thực thi (Engine) bắt đầu có dấu hiệu phản hồi chậm.

Đảm bảo khả năng mở rộng: Kiểm chứng rằng hệ thống vẫn duy trì được tốc độ xử lý ổn định khi người dùng mở rộng quy mô dự án từ vài chục lên hàng trăm Test Cases.

#### **\* Đặc điểm kiểm thử hiệu năng trong dự án Phần mềm:**

Kiểm thử dưới tải trọng thay đổi: Thực hiện kiểm thử bằng cách nạp vào hệ thống các tệp kịch bản có độ dài khác nhau (từ 10 steps đến 500 steps) để đo lường thời gian Parsing (phân tích) và thời gian Execution (thực thi).

- Giả lập việc chạy song song (Parallel execution) nhiều kịch bản để xem khả năng phân bổ luồng (threads) của Engine.

Kiểm thử tính ổn định dài hạn: Kiểm tra trạng thái của ứng dụng khi thực thi liên tục trong thời gian dài (ví dụ: chạy một bộ kịch bản kiểm thử kéo dài 4-8 tiếng). Mục đích chính là phát hiện lỗi Memory Leak (rò rỉ bộ nhớ) – một rủi ro thường gặp ở các ứng dụng phát triển trên nền tảng Electron/Node.js.

Đo lường thời gian phản hồi:

- Đo lường thời gian từ lúc gửi yêu cầu quét mã nguồn đến lúc AI Agent trả về kết quả gợi ý.
- Đánh giá độ trễ hiển thị log: Đảm bảo log được đổ ra màn hình UI ngay lập tức khi Engine thực thi mà không gây hiện tượng "nghẽn cổ chai" (bottleneck).

### **2.5.7 Kiểm thử bảo mật**

\* Mục đích

Kiểm thử bảo mật trong Phần mềm nhằm đảm bảo rằng các dữ liệu nhạy cảm của người dùng (tài khoản kiểm thử, API Keys, mật khẩu môi trường) được bảo vệ an toàn và ứng dụng không bị lợi dụng để tấn công vào máy tính của người dùng hoặc phần mềm mục tiêu. Mục tiêu chính bao gồm:

Bảo vệ thông tin người dùng: Đảm bảo các cấu hình nhạy cảm được mã hóa hoặc che giấu, không bao giờ lộ dưới dạng văn bản thuần trên giao diện hoặc tệp log.

Ngăn chặn hành vi vượt quyền: Đảm bảo các kịch bản kiểm thử không thể truy cập trái phép vào các tệp tin hệ thống quan trọng ngoài phạm vi Workspace được cấp quyền.

Đánh giá rủi ro: Xác định các điểm yếu trong cấu hình hoặc logic xử lý để ngăn chặn việc ứng dụng bị lợi dụng làm bàn đạp cho các cuộc tấn công bên ngoài.

\* Đặc điểm kiểm thử bảo mật trong dự án Phần mềm:

Mô phỏng tấn công nội bộ & cấu hình: Thay vì tập trung vào XSS hay SQL Injection, kiểm thử bảo mật cho Phần mềm tập trung vào các rủi ro cụ thể như:

- Kiểm thử Sandbox: Mô phỏng việc một kịch bản kiểm thử cố tình thực thi mã độc (ví dụ: lệnh xóa tệp hệ thống) để xác định xem cơ chế cô lập của ứng dụng có chặn đứng được hành vi này hay không.
- Kiểm thử rò rỉ dữ liệu: Kiểm tra xem các biến môi trường nhạy cảm (ENV\_VAR) có bị ghi lại vào tệp log của hệ thống hay không.

Kiểm thử tĩnh (Static Testing - SAST):

- Quét mã nguồn tĩnh để phát hiện các đoạn mã có lỗ hổng, các hàm nguy hiểm (như eval()) có thể bị lợi dụng để thực thi mã độc.
- Kiểm tra cấu hình Electron/Node.js để đảm bảo các quyền truy cập hệ thống đã được giới hạn ở mức tối thiểu (Least Privilege).

Kiểm thử động (Dynamic Testing - DAST):

- Theo dõi hành vi ứng dụng trong thời gian thực khi đang thực thi kiểm thử: Kiểm tra xem các kết nối mạng được khởi tạo bởi Engine có nằm trong danh sách "được phép" (Whitelist) hay không.
- Thử nghiệm nạp các tệp dự án (Workspace) độc hại hoặc sai định dạng để xem hệ thống có tự động từ chối và bảo vệ cấu trúc nội tại hay không.

Công cụ tích hợp: Sử dụng các công cụ quét mã nguồn (như Snyk, SonarQube) tích hợp vào quy trình phát triển để phát hiện sớm các lỗ hổng từ các thư viện bên thứ ba (Dependencies) mà Phần mềm đang sử dụng.

## **2.6 Điều kiện chấp nhận**

### **2.6.1 Tiêu chí hoàn thành kiểm thử**

Sản phẩm được coi là đã hoàn tất giai đoạn kiểm thử khi thỏa mãn các điều kiện sau:

Độ bao phủ kiểm thử: 100% các kịch bản kiểm thử trong phạm vi đã được thực thi.



Tỷ lệ lỗi:

- 100% các lỗi Nghiêm trọng (Critical) và Cao (High) đã được sửa và kiểm thử lại (re-test).
- Không còn lỗi nào ở mức Trung bình (Medium) tồn tại trong các luồng nghiệp vụ chính (Critical Path).

Độ ổn định: Hệ thống vượt qua được bài kiểm tra chạy liên tục (Soak Testing) trong 24 giờ mà không phát sinh hiện tượng treo hoặc rò rỉ bộ nhớ (Memory Leak).

Tài liệu: Toàn bộ báo cáo kiểm thử (Test Summary Report) và Ma trận truy xuất nguồn gốc (RTM) đã được cập nhật và phê duyệt.

## 2.6.2 Tiêu chí chấp nhận sản phẩm

Sản phẩm được khách hàng hoặc người dùng cuối chấp nhận khi:

Độ chính xác: AI Agent sinh ra được mã kịch bản (Test Code) đúng cú pháp và chạy được ngay mà không cần chỉnh sửa thủ công quá 10% tổng số bước.

Tương thích: Ứng dụng khởi chạy và thực thi được các kịch bản kiểm thử trên cả 3 hệ điều hành Windows, macOS và Linux.

Tính toàn vẹn dữ liệu: Mọi cấu hình, biến môi trường và tệp kịch bản được lưu trữ, đọc/ghi chính xác vào hệ thống tệp tin của người dùng mà không bị hỏng cấu trúc.

Trải nghiệm người dùng: Các thông báo lỗi (Validation messages) hiển thị rõ ràng, giúp người dùng dễ dàng hiểu và sửa lỗi nhập liệu trong Form cấu hình.

Hiệu năng: Thời gian phản hồi của giao diện khi thực thi test không vượt quá 2 giây, đảm bảo người dùng không có cảm giác ứng dụng bị "đóng băng".

## 2.7 Phân loại lỗi

| Mức độ | Đặc tả lỗi (Phần mềm Application) |
|--------|-----------------------------------|
|--------|-----------------------------------|

|                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Cao<br/>(Critical/High)</p> | <ul style="list-style-type: none"> <li>+ Lỗi treo toàn bộ ứng dụng (Application Crash) khi khởi động Engine.</li> <li>+ Lỗi sai lệch nghiêm trọng kết quả kiểm thử (Ví dụ: Pass nhưng thực tế Fail).</li> <li>+ Lỗi mất dữ liệu: Tập kịch bản (JSON/YAML) bị hỏng hoặc mất sau khi lưu.</li> <li>+ Lỗi bảo mật: Lộ thông tin nhạy cảm (API Key, Mật khẩu) trong file log.</li> <li>+ AI Agent bị treo hoặc không thể quét mã nguồn trong thư mục dự án.</li> </ul>                                                                                                                                                                                                                                                                                                                                                               |
| <p>Trung bình<br/>(Medium)</p> | <ul style="list-style-type: none"> <li>+ Lỗi hiển thị báo cáo: Biểu đồ kết quả không khớp với dữ liệu log thực tế.</li> <li>+ Không thể kết nối với các trình giả lập hoặc trình duyệt (Adapter connection timeout).</li> <li>+ Lỗi giao diện: Các bước bị vỡ bố cục khi kéo thả (drag-and-drop).</li> <li>+ Không lưu được cấu hình tùy chọn của người dùng trong phần Settings.</li> <li>+ AI Agent sinh ra code có lỗi cú pháp nhẹ (cần chỉnh sửa nhỏ để chạy được).</li> <li>+ Lỗi gián đoạn AI (AI Timeout/Rate Limit): Khi tính năng Autogen gửi mã nguồn cho AI phân tích nhưng bị quá tải kết nối hoặc API Key hết hạn, hệ thống Phần mềm không được treo (crash) mà phải hủy tiến trình an toàn và hiển thị thông báo: 'Dịch vụ AI đang gián đoạn, vui lòng kiểm tra lại cấu hình API Key hoặc thử lại sau'.</li> </ul> |

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Thấp (Low) | <ul style="list-style-type: none"> <li>+ Các lỗi về hiển thị (CSS/UI): Chữ bị tràn, sai font chữ hoặc màu sắc không đúng trên Light/Dark mode.</li> <li>+ Thiếu thông báo xác nhận khi thực hiện các hành động không quan trọng (ví dụ: thu gọn cây thư mục).</li> <li>+ Thông báo lỗi (Validation message) chưa rõ ràng hoặc gây khó hiểu cho người dùng.</li> <li>+ Lỗi chính tả trong giao diện người dùng (typos).</li> </ul> |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## PHẦN III: THỰC HIỆN KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

### 3.1 Cài đặt môi trường kiểm thử

#### 3.1.1 Tổng quan môi trường kiểm thử

Để đánh giá tính đúng đắn và hiệu quả của nền tảng kiểm thử tự động JKAuto, nhóm tiến hành xây dựng một môi trường thực nghiệm hoàn chỉnh bao gồm hệ thống cần kiểm thử, công cụ kiểm thử và các thành phần hỗ trợ thực thi. Môi trường này được thiết kế nhằm mô phỏng quy trình kiểm thử trong thực tế, cho phép xây dựng, thực thi và đánh giá các kịch bản kiểm thử tự động trên cả giao diện người dùng và các dịch vụ API.

Trong phạm vi đề án, hệ thống được lựa chọn để kiểm thử là **Electronic Content Management (ECM)** – một ứng dụng quản lý tài liệu điện tử được xây dựng theo mô hình Web Application. Hệ thống ECM bao gồm các chức năng nghiệp vụ điển hình như xác thực người dùng, quản lý tài liệu, quản lý thư mục, phân quyền truy cập và tìm kiếm dữ liệu. Đây là những chức năng phổ biến trong các hệ thống quản lý doanh nghiệp, phù hợp để đánh giá khả năng kiểm thử của JKAuto.

Môi trường thực nghiệm gồm ba thành phần chính hoạt động đồng thời:

| Thành phần                  | Vai trò                                                                                |
|-----------------------------|----------------------------------------------------------------------------------------|
| Hệ thống cần kiểm thử (ECM) | Cung cấp các chức năng nghiệp vụ và dịch vụ API phục vụ kiểm thử.                      |
| Giao diện người dùng Web    | Môi trường để thực hiện các kịch bản kiểm thử End-to-End mô phỏng thao tác người dùng. |
| JKAuto Desktop              | Công cụ xây dựng, quản lý và thực thi các kịch bản kiểm thử tự động.                   |

Bảng 3.1. Các thành phần trong môi trường kiểm thử

Trong quá trình thực nghiệm, JKAuto đóng vai trò là trung tâm điều phối toàn bộ quá trình kiểm thử. Người dùng có thể thiết kế Test Case, tổ chức Test Suite, cấu hình môi trường thực thi, chạy kiểm thử và xem kết quả ngay trên cùng một giao diện.

Hệ thống hỗ trợ đồng thời nhiều hình thức kiểm thử khác nhau. Trong phạm vi đồ án, hai loại kiểm thử chính được sử dụng gồm:

- **Kiểm thử API (API Testing):** Thực hiện gửi các yêu cầu HTTP đến các dịch vụ RESTful của hệ thống, sau đó kiểm tra mã trạng thái (Status Code), dữ liệu phản hồi (Response Body), Header và các điều kiện xác nhận (Assertion).
- **Kiểm thử giao diện người dùng (End-to-End Testing):** Điều khiển trình duyệt tự động nhằm mô phỏng các thao tác của người dùng cuối như đăng nhập, nhập dữ liệu, nhấn nút, điều hướng giữa các màn hình và xác minh kết quả hiển thị trên giao diện.

Việc kết hợp hai hình thức kiểm thử này giúp đánh giá đầy đủ cả tầng giao diện lẫn tầng dịch vụ của hệ thống, đồng thời phản ánh đúng quy trình kiểm thử trong môi trường phát triển phần mềm hiện nay.

### 3.1.2 Yêu cầu môi trường phần mềm

Để triển khai và vận hành JKAuto cũng như hệ thống cần kiểm thử, môi trường thực nghiệm cần đáp ứng các yêu cầu về phần mềm như trình bày trong Bảng 3.2.

| Thành phần                      | Vai trò                                                    |
|---------------------------------|------------------------------------------------------------|
| Node.js                         | Môi trường thực thi cho Engine và các dịch vụ của hệ thống |
| Package Manager (pnpm hoặc npm) | Quản lý thư viện và các gói phụ thuộc của dự án            |
| Playwright                      | Công cụ điều khiển trình duyệt phục vụ kiểm thử giao diện  |
| Trình duyệt Chromium            | Thực thi các kịch bản kiểm thử Web                         |

|              |                                     |
|--------------|-------------------------------------|
| Git          | Quản lý mã nguồn và phiên bản dự án |
| Hệ điều hành | Windows, macOS hoặc Linux           |

Bảng 3.2. Yêu cầu phần mềm

Các thành phần trên đều là các công nghệ phổ biến trong phát triển phần mềm hiện đại, có khả năng hoạt động đa nền tảng và dễ dàng tích hợp với các hệ thống CI/CD.

### 3.1.3 Yêu cầu phần cứng

Để đảm bảo quá trình thực thi kiểm thử diễn ra ổn định, đặc biệt đối với các kịch bản End-to-End sử dụng trình duyệt tự động, hệ thống cần đáp ứng một số yêu cầu tối thiểu về phần cứng như sau.

| Tài nguyên         | Tối thiểu                                | Khuyến nghị                                 |
|--------------------|------------------------------------------|---------------------------------------------|
| Bộ xử lý (CPU)     | 4 nhân                                   | 8 nhân trở lên                              |
| Bộ nhớ RAM         | 8 GB                                     | 16 GB trở lên                               |
| Dung lượng lưu trữ | 5 GB trống                               | 20 GB trống                                 |
| Mạng               | Không bắt buộc đối với môi trường cục bộ | Khuyến nghị khi sử dụng AI Agent hoặc CI/CD |

Bảng 3.3 Yêu cầu phần cứng

## 3.2 Kiểm thử Hệ thống Kiểm thử

Sau khi môi trường thực nghiệm được chuẩn bị hoàn chỉnh, nhóm tiến hành triển khai dự án kiểm thử trên nền tảng JKAuto. Quá trình triển khai bao gồm việc khởi tạo dự án, cấu hình môi trường thực thi, xây dựng các kịch bản kiểm thử, tổ chức Test Suite và thiết lập

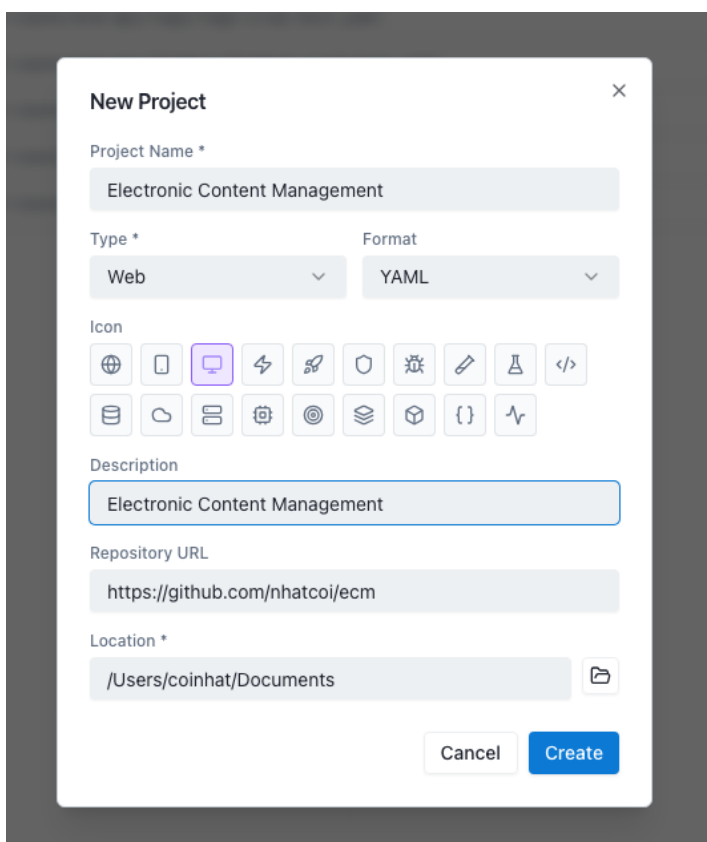
quy trình thực thi. Các bước này tạo thành nền tảng để tiến hành thực nghiệm và đánh giá hệ thống ở các mục tiếp theo.

### 3.2.1 Khởi tạo dự án kiểm thử

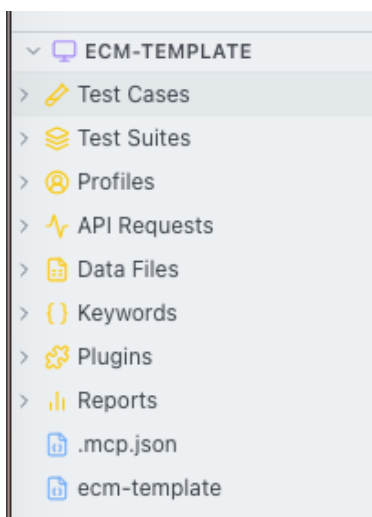
Đầu tiên, một dự án kiểm thử mới được tạo trong JKAuto Desktop thông qua chức năng **New Project**. Người dùng khai báo các thông tin cơ bản như tên dự án, loại ứng dụng cần kiểm thử, vị trí lưu trữ và định dạng dữ liệu của dự án.

Sau khi hoàn thành quá trình khởi tạo, JKAuto tự động sinh cấu trúc thư mục chuẩn phục vụ cho việc quản lý các thành phần kiểm thử. Các thư mục được phân chia theo từng loại tài nguyên như Test Case, Test Suite, Object Repository, API Request và Environment Profile.

Việc chuẩn hóa cấu trúc dự án ngay từ đầu giúp người phát triển dễ dàng quản lý các kịch bản kiểm thử, đồng thời tạo điều kiện thuận lợi cho việc mở rộng dự án hoặc làm việc theo nhóm.



Hình 3.1. Hộp thoại tạo dự án kiểm thử mới trong JKAuto.

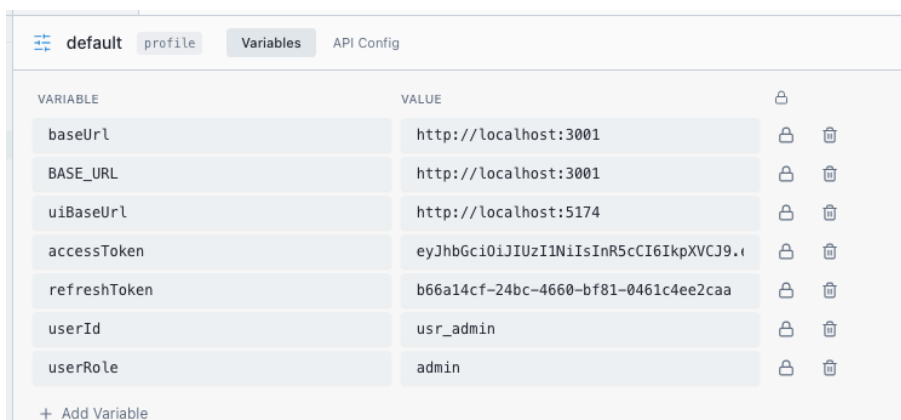


Hình 3.2. Cấu trúc dự án sau khi được khởi tạo trong Explorer.

### 3.2.2 Cấu hình môi trường thực thi

Sau khi khởi tạo dự án, người dùng tiến hành cấu hình **Environment Profile** để lưu trữ các thông tin dùng chung trong quá trình kiểm thử như biến môi trường, thông tin xác thực và các tham số thực thi.

Việc tách riêng Environment khỏi Test Case giúp cùng một kịch bản kiểm thử có thể được thực thi trên nhiều môi trường khác nhau mà không cần chỉnh sửa nội dung. Trước khi chạy kiểm thử, người dùng chỉ cần lựa chọn Profile phù hợp.



Hình 3.3. Giao diện cấu hình Environment Profile.



### 3.2.3 Xây dựng Test Case

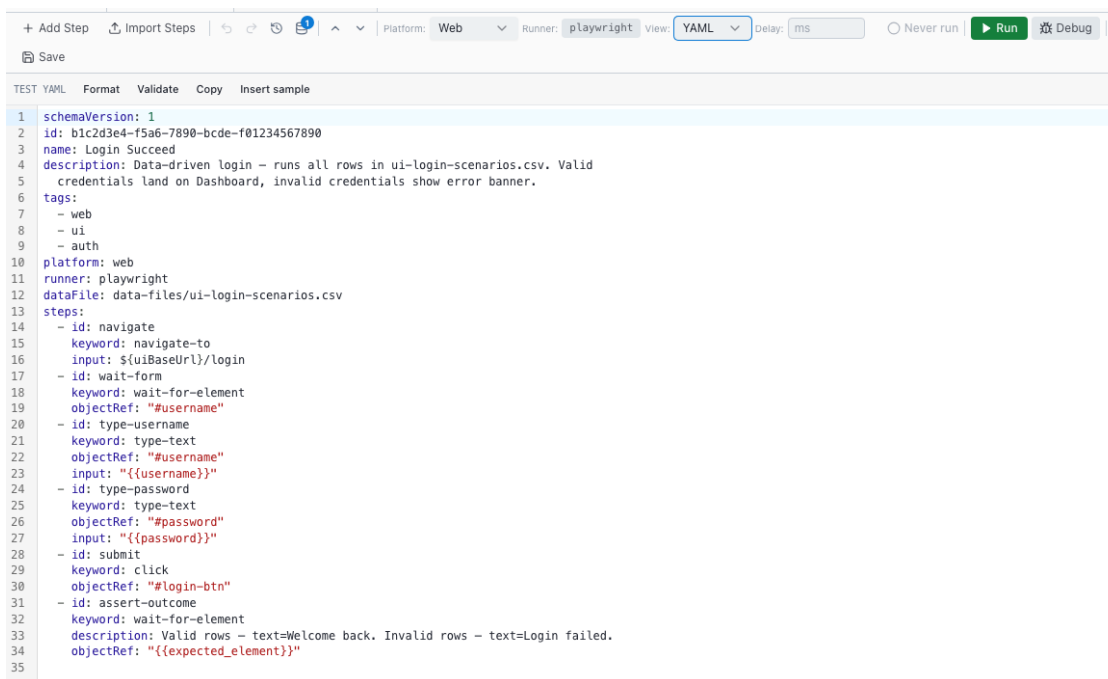
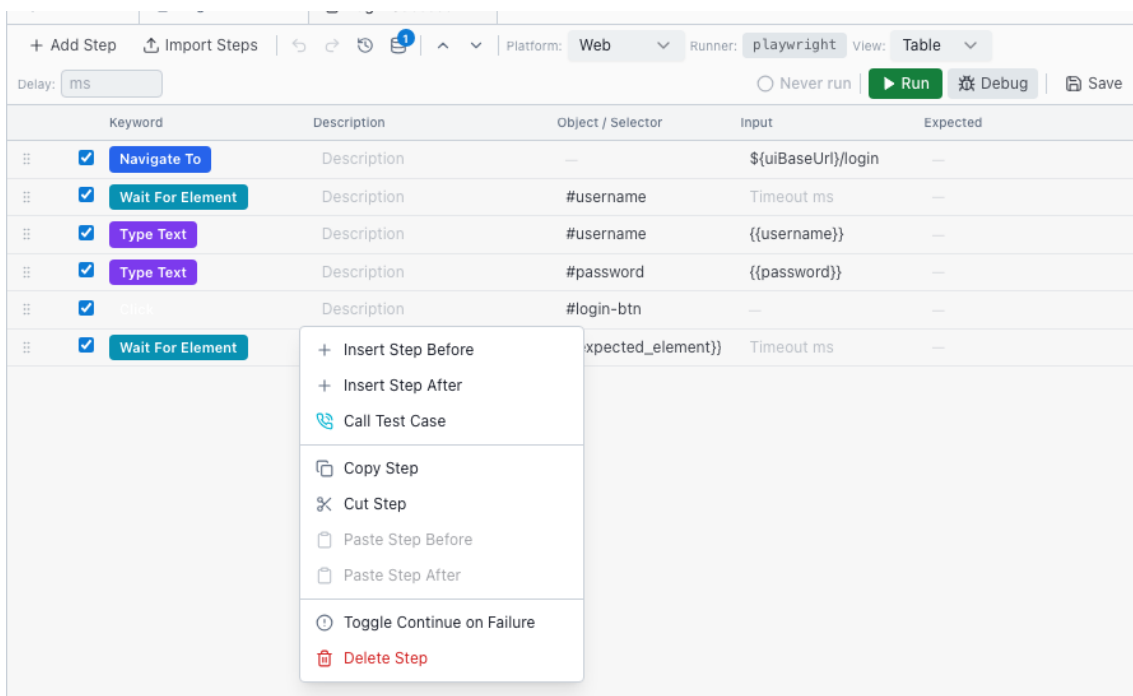
Sau khi hoàn tất cấu hình môi trường, các kịch bản kiểm thử được xây dựng bằng **Test Case Editor**.

Mỗi Test Case được mô hình hóa dưới dạng một chuỗi các bước kiểm thử (Step), trong đó mỗi Step tương ứng với một Keyword cụ thể như mở trang, nhập dữ liệu, nhấn nút, gửi yêu cầu API hoặc xác minh kết quả.

Thông qua giao diện trực quan, người dùng có thể:

- Thêm hoặc xóa Step.
- Sắp xếp lại thứ tự thực hiện.
- Lựa chọn Keyword từ thư viện.
- Khai báo dữ liệu đầu vào.
- Thiết lập điều kiện xác nhận (Assertion).
- Liên kết với Object Repository.
- Cấu hình dữ liệu sử dụng từ Environment.

Toàn bộ Test Case được lưu dưới dạng tệp cấu hình trong dự án và có thể tái sử dụng ở nhiều Test Suite khác nhau.



Hình 3.4. Giao diện Test Case Editor.

### 3.2.4 Thiết kế Test Suite

Sau khi xây dựng các Test Case riêng lẻ, nhóm tiến hành tổ chức chúng thành các Test Suite.

Một Test Suite bao gồm nhiều Test Case được sắp xếp theo một trình tự thực thi xác định. Người dùng có thể lựa chọn các Test Case cần chạy, thay đổi thứ tự thực hiện, bật hoặc tắt từng Test Case và cấu hình các tùy chọn như Continue on Failure hoặc Shared Browser Session.

Việc tổ chức thành Test Suite giúp tự động hóa các quy trình kiểm thử lớn, đồng thời hỗ trợ thực hiện Regression Test hoặc Smoke Test chỉ với một thao tác.

| Run                                 | #  | Test Case                                               | Path                                                            |
|-------------------------------------|----|---------------------------------------------------------|-----------------------------------------------------------------|
| <input checked="" type="checkbox"/> | 1  | UI - Auth guard redirects unauthenticated user to login | test-cases/ecm-ui/auth/ui-auth-guard-redirect.test.yaml         |
| <input checked="" type="checkbox"/> | 2  | UI - Login with valid credentials                       | test-cases/ecm-ui/auth/ui-login-valid.test.yaml                 |
| <input checked="" type="checkbox"/> | 3  | ui-login-with-wrong-password-shows-error                | test-cases/ecm-ui/auth/ui-login-with-wrong-password-shows-er... |
| <input checked="" type="checkbox"/> | 4  | UI - Login with empty fields blocked by form validation | test-cases/ecm-ui/auth/ui-login-empty-submit.test.yaml          |
| <input checked="" type="checkbox"/> | 5  | UI - Register new account and redirect to login         | test-cases/ecm-ui/auth/ui-register-success.test.yaml            |
| <input checked="" type="checkbox"/> | 6  | UI - Register with mismatched passwords shows error     | test-cases/ecm-ui/auth/ui-register-password-mismatch.test.ya... |
| <input checked="" type="checkbox"/> | 7  | UI - Create new document via modal                      | test-cases/ecm-ui/documents/ui-documents-create.test.yaml       |
| <input checked="" type="checkbox"/> | 8  | UI - Filter documents by status                         | test-cases/ecm-ui/documents/ui-documents-filter.test.yaml       |
| <input checked="" type="checkbox"/> | 9  | UI - Create then delete a document                      | test-cases/ecm-ui/documents/ui-documents-delete.test.yaml       |
| <input checked="" type="checkbox"/> | 10 | UI - Create new root folder                             | test-cases/ecm-ui/folders/ui-folders-create.test.yaml           |
| <input checked="" type="checkbox"/> | 11 | UI - Create nested folder under parent                  | test-cases/ecm-ui/folders/ui-folders-nested.test.yaml           |
| <input checked="" type="checkbox"/> | 12 | UI - Create new tag                                     | test-cases/ecm-ui/tags/ui-tags-create.test.yaml                 |
| <input checked="" type="checkbox"/> | 13 | UI - Search returns results for existing term           | test-cases/ecm-ui/search/ui-search-find.test.yaml               |
| <input checked="" type="checkbox"/> | 14 | UI - Clicking search result navigates to correct page   | test-cases/ecm-ui/search/ui-search-navigate.test.yaml           |

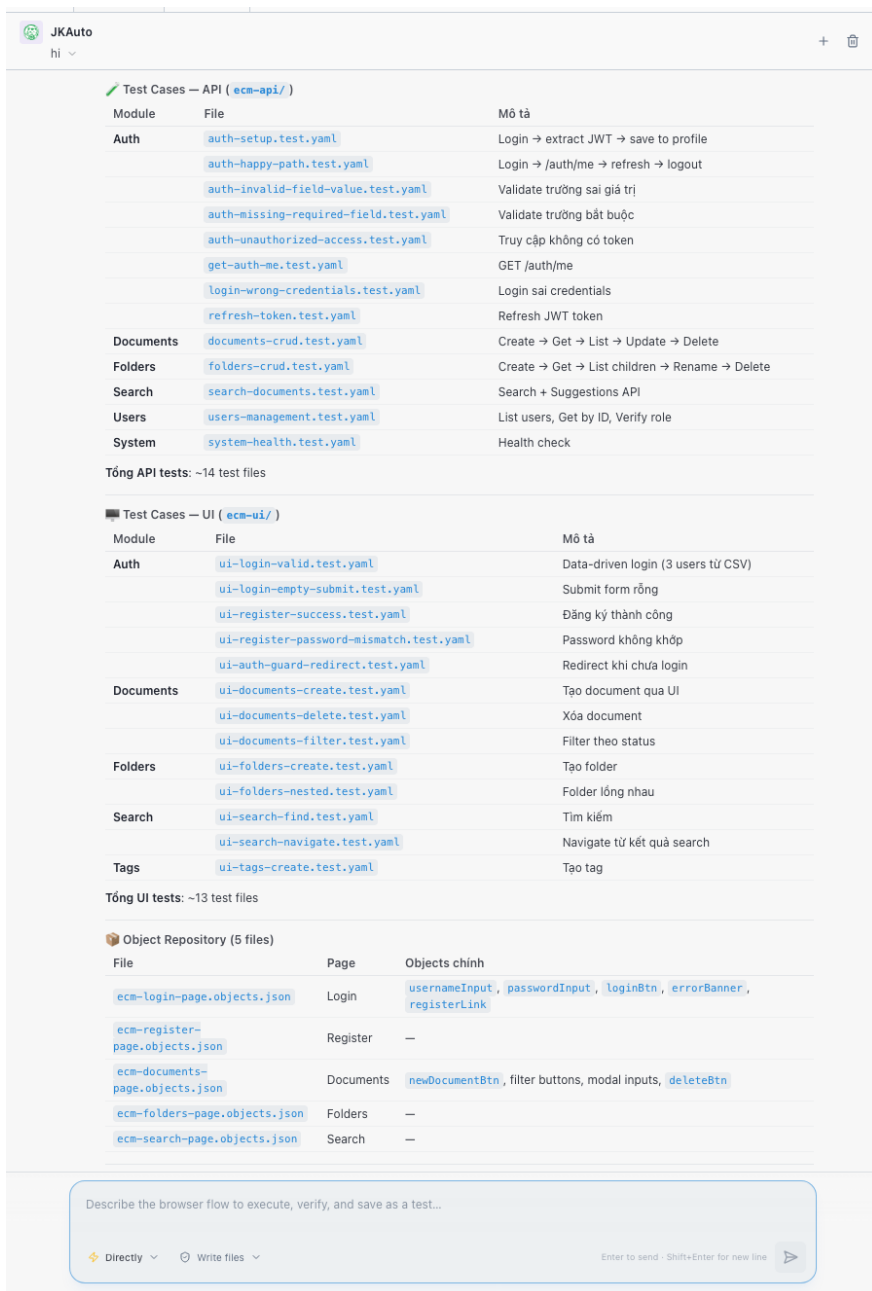
- 
- Test Suites**
- ECM API - Auth Full Regression
  - ECM API - Auth Security & Negative
  - ECM API - Documents CRUD
  - ECM API - Folders CRUD
  - ECM API - Full Regression
  - ECM API - Smoke
  - ECM API - System Health & Stats
  - ECM UI - Auth Full Regression
  - ECM UI - Auth Security & Validation
  - ECM UI - Documents Full Regression
  - ECM UI - Folders Full Regression
  - ECM UI - Full Regression
  - ECM UI - Search & Navigation
  - ECM UI - Smoke

Hình 3.5. Giao diện Test Suite Editor.

### 3.2.5 AI Agent hỗ trợ sinh kiểm thử

Tích hợp **AI Agent** hỗ trợ phân tích dự án và tự động sinh kịch bản kiểm thử thông qua giao diện Chat. Hệ thống cung cấp hai chế độ làm việc:

- **Normal:** Sinh Test Case dựa trên cấu trúc và mã nguồn của dự án.
- **Directly:** Sử dụng **Agent Loop Harness** để trực tiếp kiểm thử ứng dụng, tự đánh giá kết quả và lặp lại quá trình cho đến khi hoàn thiện bộ kiểm thử.

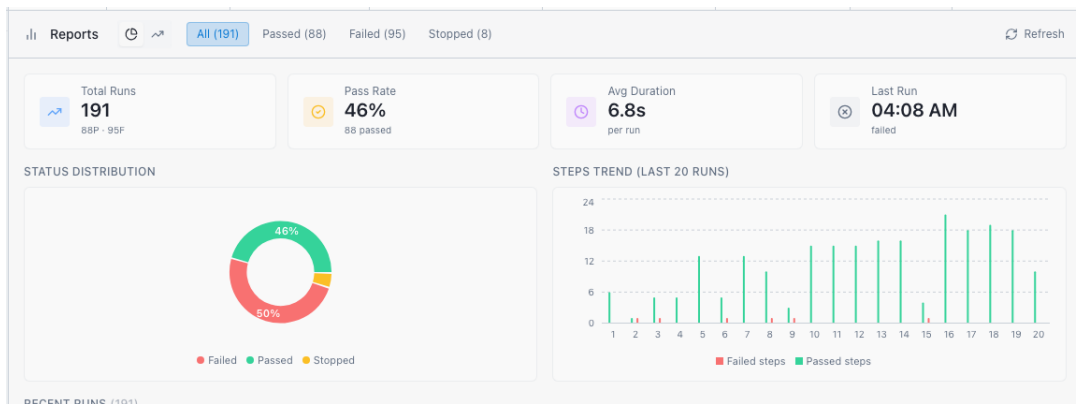


Hình 3.6. Giao diện AI Agent Chat hỗ trợ phân tích dự án và sinh kiểm thử.

### 3.2.6 Báo cáo kiểm thử

Sau khi hoàn thành quá trình thực thi, JKAuto tự động tổng hợp và lưu trữ kết quả kiểm thử trong **Reports**. Báo cáo cung cấp thông tin về trạng thái của từng Test Case, số lượng bước thực hiện, thời gian chạy và các lỗi phát sinh (nếu có).

Ngoài việc xem trực tiếp trên giao diện, kết quả kiểm thử còn có thể được xuất ra các định dạng phổ biến để phục vụ lưu trữ, chia sẻ và tích hợp với các hệ thống CI/CD.



Hình 3.7. Giao diện Reports hiển thị kết quả kiểm thử.

## PHẦN IV: BÁO CÁO PHÂN TÍCH TỔNG HỢP

### 4.1 Tổng hợp kết quả kiểm thử

Sau khi thực hiện kiểm thử, JKAuto tự động tổng hợp kết quả từ các Test Case và Test Suite, cung cấp các thống kê về trạng thái thực thi, số lượng bước kiểm thử, thời gian thực hiện và tỷ lệ thành công.

**Bảng 4.1. Tổng hợp kết quả kiểm thử**

| Phân loại        | Test Case | Passed    | Failed   | Tổng Step | Tỷ lệ Pass  |
|------------------|-----------|-----------|----------|-----------|-------------|
| E2E Web          | 4         | 4         | 0        | 48        | 100%        |
| API              | 6         | 6         | 0        | 6         | 100%        |
| <b>Tổng cộng</b> | <b>10</b> | <b>10</b> | <b>0</b> | <b>54</b> | <b>100%</b> |

Tổng thời gian thực thi của toàn bộ Smoke Suite khoảng **14 giây**.

Hình 4.1. Dashboard tổng hợp kết quả kiểm thử.

### 4.2 Đánh giá chất lượng hệ thống

Kết quả thực nghiệm được đánh giá theo một số tiêu chí của tiêu chuẩn ISO/IEC 25010.

**Bảng 4.2. Đánh giá chất lượng hệ thống**

| Đặc tính          | Kết quả |
|-------------------|---------|
| Phù hợp chức năng | Đạt     |
| Hiệu năng         | Đạt     |

|                      |     |
|----------------------|-----|
| Độ tin cậy           | Đạt |
| Bảo mật              | Đạt |
| Khả năng sử dụng     | Đạt |
| Khả năng tự động hóa | Đạt |

### 4.3 Cấu trúc mã nguồn

Mã nguồn dự án được quản lý trên GitHub tại: <https://github.com/nhatcoi/jkauto>

### 4.4 Kết luận

Kết quả thực nghiệm cho thấy JKAuto hỗ trợ hiệu quả quá trình kiểm thử tự động đối với hệ thống ECM. Toàn bộ 10 Bộ Test Case được thực thi thành công với tỷ lệ 100% Passed, đáp ứng các yêu cầu kiểm thử đã đặt ra.

## TÀI LIỆU THAM KHẢO

- [1] Playwright Official Documentation. (2026). Playwright: Fast and reliable end-to-end testing for modern web apps. Truy cập tại: <https://playwright.dev/>
- [2] Appium Documentation. (2026). Appium: Automation for mobile apps. Truy cập tại: <https://appium.io/docs/en/>
- [3] Roger S. Pressman. (2014). Software Engineering: A Practitioner's Approach. McGraw-Hill Education. (Tài liệu nền tảng về quy trình kiểm thử phần mềm).
- [4] Rex Black. (2012). Foundations of Software Testing: ISTQB Certification. Cengage Learning. (Các chuẩn mực quốc tế về ISTQB trong kiểm thử).
- [5] Jest/Vitest Framework. Unit testing framework documentation. Truy cập tại: <https://vitest.dev/>
- [6] Node.js Foundation. Node.js Desktop Application Development with Electron. Truy cập tại: <https://www.electronjs.org/docs/latest/>